



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Informática

Mención en Sistemas de Información

Evaluación y retroalimentación automática de ejercicios de programación mediante VPL y modelos LLM

Automatic evaluation and feedback of programming exercises using VPL and LLM models

Realizado por

Manuel Valdivielso Rodríguez

Tutorizado por

Rafael Marcos Luque Baena

Co-tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

Málaga, November 25, 2025



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA

GRADO EN INGENIERÍA INFORMÁTICA

Mención en Sistemas de Información

**Evaluación y retroalimentación automática de
ejercicios de programación mediante VPL y modelos
LLM**

**Automatic evaluation and feedback of programming
exercises using VPL and LLM models**

Realizado por

Manuel Valdivielso Rodríguez

Tutorizado por

Rafael Marcos Luque Baena

Co-tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, NOVEMBER 25, 2025

Fecha defensa: diciembre de 2025

Resumen

La programación es una asignatura fundamental en la mayoría de los grados con orientación tecnológica y resulta indispensable en los estudios de Informática. Para dominarla es necesaria una práctica constante mediante ejercicios. Obtener una retroalimentación rápida sobre los errores cometidos es clave para la mejora del aprendizaje. Sin embargo, la corrección de todas las prácticas del alumnado supone un gran esfuerzo y un considerable consumo de tiempo [3].

En la actualidad, los docentes ya cuentan con herramientas de corrección automática de caja negra, como VPL (Virtual Programming Lab), para facilitar la corrección de ejercicios. VPL es especialmente útil, ya que está integrado en Moodle, el sistema base del Campus Virtual. No obstante, la carga de trabajo sigue siendo considerable si se busca proporcionar una retroalimentación más detallada sobre el código.

Una posible solución para abordar este problema sería desarrollar un software capaz de analizar el código internamente para cada ejercicio propuesto por el profesor. Sin embargo, esta opción resulta poco viable debido a la gran variedad de enfoques y estructuras de programación que pueden emplearse. Al mismo tiempo, la reciente aparición de modelos de lenguaje de gran tamaño (LLMs), como ChatGPT, ha abierto la puerta al uso de estos sistemas para un análisis de código más avanzado.

Por consiguiente, este TFG se enfoca en el desarrollo de una solución que conecta VPL con un modelo LLM para realizar un análisis automatizado del código enviado por los alumnos y apoyar la corrección automática. La evaluación se basa en su idoneidad según los criterios establecidos para cada ejercicio. En la solución implementada se despliega una API en un servidor de la universidad que consume modelos LLM a través de un proveedor en la nube.

Palabras clave: Programación, corrección automática, VPL, LLM.

Abstract

Programming is a fundamental subject in most technology-oriented degree programmes and is essential in Computer Science studies. To master it, constant practice through exercises is necessary. Receiving quick feedback on the mistakes made is key to improving learning. However, correcting all student assignments requires a major effort and a considerable investment of time [3].

At present, teachers already have black-box automatic assessment tools, such as VPL (Virtual Programming Lab), to facilitate the grading of exercises. VPL is particularly useful because it is integrated into Moodle, the core system of the Virtual Campus. Nevertheless, the workload remains considerable if the goal is to provide more detailed feedback on the code.

One possible way to address this problem would be to develop software capable of internally analysing the code for each exercise proposed by the instructor. However, this option is not very feasible due to the wide variety of programming approaches and structures that can be used. At the same time, the recent emergence of large language models (LLMs), such as ChatGPT, has opened the door to using these systems for more advanced code analysis.

Therefore, this final degree project focuses on developing a solution that connects VPL with an LLM to perform an automated analysis of the code submitted by students and support automatic assessment. The evaluation is based on its adequacy according to the criteria established for each exercise. In the implemented solution, an API is deployed on a university server that consumes LLMs through a cloud provider.

Keywords: Programming, automatic assessment, VPL, LLM.

Resumen	1
Abstract	2
1 Introducción	7
1.1 Motivación	7
1.2 Objetivos	8
1.3 Tecnologías usadas	9
1.3.1 Python	9
1.3.2 FastAPI	9
1.3.3 Visual Studio Code	10
1.3.4 Uvicorn	10
1.3.5 VPL	11
1.3.6 OpenRouter	11
1.3.7 Mistral	11
1.3.8 Bash	12
1.3.9 PuTTY	12
1.4 Metodología del trabajo	13
1.5 Estructura del documento	14
2 Estado del arte y marco tecnológico	15
2.1 Estado del arte	15
2.1.1 VPL	15
2.1.2 LLMs en evaluación de código	16
2.2 Marco tecnológico	19
2.2.1 Stack seleccionado	19
2.2.2 Justificación de decisiones	20
2.2.3 Condicionantes y versiones	21
3 Análisis de requisitos	23
3.1 Requisitos funcionales	23
3.2 Requisitos no funcionales	24
4 Diseño de la arquitectura del sistema	26
4.1 Visión general	26
4.2 Componentes del sistema	27
4.3 Estructura y flujo del sistema	29
5 Implementación e integración	33

5.1	Desarrollo conjunto de la API y del script de VPL	33
5.1.1	Primera versión: prototipo local con LM Studio y envío de código	34
5.1.2	Segunda versión: entrada estructurada y adaptación del script de VPL	34
5.1.3	Tercera versión: proveedor Groq en servidor de la UMA	35
5.1.4	Cuarta versión: migración a OpenRouter y reorganización del flujo en VPL	35
5.1.5	Quinta versión: integración final con BIOTES, restricciones y notas combinadas	36
5.1.6	Pruebas básicas en aislamiento	37
5.2	Pruebas de la integración VPL–API	38
5.2.1	PI1: Ejecución del tester y nota de tests	39
5.2.2	PI2: Construcción del JSON con contexto completo	39
5.2.3	PI3: Integración independiente del lenguaje de programación	39
5.2.4	PI4: Cálculo y devolución de las notas	40
5.2.5	PI5: tolerancia a fallos en la llamada a la API	41
6	Pruebas de retroalimentación y análisis	42
6.1	Metodología de las pruebas	42
6.2	Resultados y evidencias	43
6.2.1	Estudio de caso 1: ejercicio sin discrepancias relevantes	43
6.2.2	Estudio de caso 2: ejercicio con problemas en la corrección de las restricciones	50
6.3	Análisis de los errores observados	53
6.4	Discusión y posibles mejoras	54
7	Conclusiones y líneas futuras	56
7.1	Conclusiones	56
7.2	Líneas Futuras	57
	Apéndice A. Manual de Instalación	58
A.1	Requisitos previos	58
A.2	Preparación de la máquina	58
A.2.1	Crear el directorio del proyecto	58
A.2.2	Crear y activar el entorno virtual	59
A.2.3	Crear el archivo .env con la API Key	59
A.2.4	Ejecutar la API	59
A.2.5	Comprobar el funcionamiento básico de la API	60
A.3	Configuración de la actividad VPL en Moodle	60
A.3.1	Creación de la actividad	60
A.3.2	Añadir los ficheros necesarios	60
A.3.3	Contenido del script vp_evaluate.sh	61
A.4	Prueba de funcionamiento	61

Apéndice B. Manual de Usuario	62
B.1 Introducción	62
B.2 Perfil del profesor	62
B.2.1 Creación de una actividad VPL en Moodle	62
B.2.2 Ficheros del sistema comunes	62
B.2.3 Ficheros específicos de cada ejercicio	63
B.2.4 Puesta en marcha de la API con uunicorn	64
B.2.5 Comprobación rápida del sistema	64
B.3 Perfil del estudiante	65
B.3.1 Acceso a la actividad y entrega	65
B.3.2 Edición y reevaluación	65
B.3.3 Evaluación y retroalimentación	65
B.4 Problemas frecuentes y recomendaciones	66
B.4.1 Para el profesor	66
B.4.2 Para el estudiante	66

1.1	Logo de Python	9
1.2	Logo de FastApi	10
1.3	Logo de Visual Studio Code	10
1.4	Logo de Uvicorn	11
1.5	Logo de OpenRouter	11
1.6	Logo de OpenRouter	12
1.7	Logo de Mistral	12
1.8	Logo de Bash	12
1.9	Logo de PuTTY	13
2.1	Logo de Mistral	17
2.2	Logo de Deepseek	18
2.3	Logo de Llama (Meta)	18
2.4	Logo de GPT	18
4.1	Estructura conceptual	32
5.1	Respuesta correcta del <i>endpoint /health</i> en local	38
5.2	Prueba mínima del <i>endpoint /retroalimentacion</i> en local	38
5.3	Resultado de <i>BIOTES</i> se envía a la API y se convierte en nota de tests	39
5.4	<i>JSON</i> enviado por el <i>script</i> con el contexto del ejercicio	40
5.5	Evaluación correcta de un ejercicio Java	40
5.6	Desglose de nota de tests, nota de la IA y nota final en VPL	40
5.7	Fallo en la API: mensaje de error mostrado en la salida de VPL	41
6.1	Código de la solución 1	44
6.2	Código de la solución 2	45
6.3	Captura de la solución 3	46
6.4	Captura de la solución correcta	51

1

Introducción

1.1 Motivación

La enseñanza de la programación presenta un desafío constante: lograr que el alumnado no solo ejecute código que funcione, sino que entienda los conceptos, aprenda de sus errores y mejore su forma de razonar. En la práctica, la retroalimentación que recibe el estudiante desempeña un papel decisivo en este proceso de aprendizaje. Sin embargo, cuando los grupos de clase son numerosos, ofrecer comentarios individualizados se convierte en una tarea enormemente compleja para el profesorado.

En los últimos años, las universidades han incorporado sistemas de **corrección automática** mediante *Moodle* y su complemento *VPL (Virtual Programming Lab)*, que permiten evaluar de manera objetiva el código del alumnado. Estos entornos verifican si el programa cumple con las pruebas establecidas y generan una nota basada en los resultados. Aunque esto agiliza la evaluación y garantiza la objetividad, deja de lado el componente formativo: el sistema informa de si un programa funciona o no, pero no explica al estudiante por qué ha fallado ni cómo puede mejorar. Como consecuencia, el alumnado recibe un número o un porcentaje, pero no una orientación clara que le ayude a progresar.

Esta carencia de explicaciones personalizadas provoca que muchos estudiantes perciban la programación como un proceso mecánico de ensayo y error, en lugar de una disciplina en la que se aprende a razonar, estructurar y escribir código de forma correcta. Al mismo tiempo, el profesorado se ve obligado a invertir un tiempo considerable en revisar manualmente los ejercicios para ofrecer una retroalimentación significativa, algo difícil de sostener cuando hay decenas o cientos de entregas por cada práctica.

El avance de la **Inteligencia Artificial** en los últimos años abre una oportunidad única para mejorar este proceso. Los *modelos de lenguaje de gran tamaño* actuales son capaces de comprender texto, analizar código y redactar explicaciones comprensibles. Aplicar estas tecnologías al ámbito educativo puede ayudar a generar comentarios automáticos que sean coherentes, formativos y adaptados a cada estudiante, sin perder la objetividad que ofrecen las pruebas automáticas ya existentes.

Por todo ello, este proyecto nace con la motivación de mejorar la calidad del aprendizaje en las asignaturas de programación. Su propósito no es sustituir al profesorado, sino proporcionar una herramienta que complemente su trabajo, ofrezca al alumnado

una guía más clara y fomente la comprensión profunda de los errores. En definitiva, se busca aprovechar el potencial de la inteligencia artificial para hacer que la evaluación automática sea no solo un proceso de calificación, sino también una oportunidad real para aprender.

1.2 Objetivos

El objetivo principal de este proyecto de fin de grado es desarrollar un sistema que, a partir del código entregado por el alumno y las instrucciones del docente, genere retroalimentación rápida, clara y personalizada sobre la solución y verifique el cumplimiento de los requisitos del enunciado. VPL actuará como verificador automático mediante casos de prueba (caja negra), mientras que un modelo LLM realizará una revisión estática guiada por el enunciado y colaborará en la estimación de una nota sobre 10.

Además, el proyecto tiene los siguientes objetivos específicos:

- **Integración técnica.** Diseñar e implementar una API (FastAPI) que orqueste la consulta a un LLM público (Open-Router/Mistral) desde VPL, incluyendo autenticación y manejo de errores. Definir el contrato JSON (campos obligatorios/opcionales), cabeceras y control de CORS; devolver mensajes de error claros (4xx/5xx) y registrar trazas para depuración. Proteger la API key (variables de entorno, `.env`).
- **Retroalimentación pedagógica.** Generar comentarios dirigidos al estudiante con formato compacto (líneas cortas), tono docente y consejos accionables. Estandarizar el formato (resumen, estimación de casos, consejos, buenas prácticas, próximos pasos) y una nota estimada 0–10 que combine el resultado de los casos de prueba y la valoración del LLM respecto al encaje con el enunciado. Limitar alucinaciones con un *prompt* claro (no inferir ejecuciones), temperatura baja y rúbrica explícita.
- **Multilinguaje.** Diseñar y validar la integración para que funcione con varios lenguajes de programación (Python, C/C++, Java, JavaScript), detectando automáticamente el fichero de código del alumno y construyendo el comando de ejecución adecuado en cada caso. Comprobar que la API trata el código como texto independiente del lenguaje y maneja correctamente codificaciones, tamaños de entrada y, cuando proceda, varios ficheros.
- **Integración con VPL.** Integrar la solución en el flujo normal de VPL mediante los *scripts* de evaluación, de forma que el sistema llame a la API sin que el profesorado tenga que cambiar su forma de trabajar. Garantizar que solo se ejecuta cuando el alumno pulsa *Evaluate*, que la llamada se realiza una única vez por entrega y que VPL puede seguir corrigiendo con los casos de prueba definidos por el profesor. Todo ello respetando las limitaciones de red del entorno de ejecución aislado de VPL (el *jail*) y construyendo de forma robusta el JSON a partir del enunciado, el código base, los casos de prueba y los resultados de ejecución.
- **Evaluación y mejora.** Diseñar un plan de experimentación con ejercicios reales para medir la claridad y utilidad de la retroalimentación y refinar los prompts con evidencia.
- **Despliegue y operación.** Definir un procedimiento reproducible para desplegar la API de corrección en el servidor de la UMA e integrarla con VPL, documentando la configuración de las variables de entorno (fichero `.env`), el arranque del

servicio con *uvicorn*, el uso del endpoint */health* y las comprobaciones básicas de funcionamiento (puerto accesible y ejecución de una actividad de prueba en VPL).

- **Documentación y transferencia.** Entregar guías técnicas y de uso (docente/alumno), con *scripts* reproducibles y anexos de configuración, para facilitar la adopción del sistema en otras asignaturas. Describir de forma detallada cómo se integra la API con VPL, el flujo completo de corrección (ejecución de *BIOTES*, llamada a la API y devolución de la retroalimentación) y las principales decisiones de diseño y limitaciones, de modo que otro docente pueda reproducir el despliegue y adaptar el sistema a su contexto.

1.3 Tecnologías usadas

Las tecnologías utilizadas en este proyecto son las siguientes: Python, como lenguaje de programación principal, FastAPI como framework web, Uvicorn como servidor ASGI (interfaz asíncrona entre servidores y aplicaciones Python), VPL, OpenRouter y un modelo de la familia Mistral como soporte LLM, y Visual Studio Code como editor de código. Además, se realiza el estudio de otras familias de LLMs, como DeepSeek, modelos basados en Llama y modelos basados en GPT, para fundamentar la elección del modelo empleado.

1.3.1 Python

Python es un lenguaje de programación interpretado, de alto nivel y multipropósito [17]. Se destaca por su sintaxis sencilla y legible, lo que lo hace fácil de aprender y usar [7]. Es un lenguaje de tipado dinámico, lo que significa que no es necesario declarar el tipo de las variables, y además es multiplataforma, por lo que puede ejecutarse en Windows, Linux y macOS sin modificaciones.

Es ampliamente utilizado en distintos campos como el desarrollo web, la ciencia de datos, la inteligencia artificial y la automatización [24]. Gracias a su gran ecosistema de librerías, permite trabajar con frameworks como Django y **FastAPI** para desarrollo web, o con TensorFlow y PyTorch en inteligencia artificial.



Figura 1.1. Logo de Python

1.3.2 FastAPI

FastAPI es un framework moderno y de alto rendimiento para desarrollar APIs con Python [18]. Destaca por su rapidez, facilidad de uso y su capacidad de generar documentación automática basada en OpenAPI y JSON Schema. Se basa en Starlette para

la gestión de peticiones, en Pydantic para la validación de datos y en Uvicorn como servidor ASGI de alto rendimiento, lo que garantiza seguridad y eficiencia. Además, ofrece validación automática de datos mediante anotaciones de tipo de Python, reduciendo errores en el desarrollo.

FastAPI es ampliamente utilizado en el desarrollo de microservicios, APIs REST y aplicaciones de alta concurrencia, gracias a su velocidad comparable con frameworks como Node.js y Go [23].



Figura 1.2. Logo de FastApi

1.3.3 Visual Studio Code

Visual Studio Code (VS Code) es un editor de código fuente desarrollado por Microsoft, de código abierto y multiplataforma. Su diseño ligero y modular permite trabajar con distintos lenguajes de programación mediante un sistema de extensiones [10].

Cuenta con herramientas como depuración integrada, control de versiones con Git, autocompletado inteligente (IntelliSense) y terminal incorporada. Aunque no es un IDE, puede configurarse como tal mediante extensiones. Es ampliamente utilizado en desarrollo web, programación en Python, JavaScript y otros lenguajes, así como en entornos de desarrollo con Docker y Jupyter Notebooks [25].

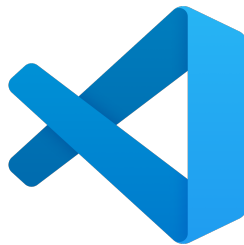


Figura 1.3. Logo de Visual Studio Code

1.3.4 Uvicorn

Uvicorn es un servidor ASGI para aplicaciones Python [20]. Está orientado a baja latencia y alta concurrencia, con soporte asíncrono y compatibilidad con HTTP/1.1 y WebSockets. Resulta adecuado para exponer APIs que necesitan responder con rapidez y gestionar múltiples conexiones simultáneas.

En este proyecto se usa para servir la API de FastAPI en el servidor de la universidad. Se configura el host y el puerto, la

carga de variables desde archivo de entorno y la ejecución en segundo plano con herramientas del sistema. Su sencillez de despliegue facilita la integración con VPL y la depuración durante el desarrollo.



Figura 1.4. Logo de Uvicorn

1.3.5 VPL

VPL (Virtual Programming Lab) es un módulo de Moodle orientado a la evaluación automática de ejercicios de programación. Ejecuta el código del alumnado en un sistema de *jail*, es decir, en un entorno aislado y controlado que limita recursos y acceso al sistema anfitrión, aumentando la seguridad y la reproducibilidad de las pruebas.

En este proyecto, VPL se integra con *BIOTES*, un tester que ejecuta los casos de prueba definidos y genera un log con el resultado de cada caso. El script `vpL_evaluate.sh` actúa como puente: selecciona el archivo de código del alumno, lanza *BIOTES* o un *fallback* en *Bash* cuando es necesario y construye el contexto (enunciado, código base, restricciones y evidencias de ejecución) que se envía a la API de corrección.



Figura 1.5. Logo de OpenRouter

1.3.6 OpenRouter

OpenRouter es una pasarela que unifica el acceso a modelos **LLM** mediante una API común [15]. Permite seleccionar el modelo, gestionar cabeceras de atribución y controlar parámetros como la temperatura y la longitud de salida. Centralizar la llamada simplifica el cambio de modelo si fuera necesario, manteniendo el mismo contrato de cliente [16].

1.3.7 Mistral

Mistral es una familia de modelos **LLM** orientada a razonamiento y programación [12]. En este proyecto se utiliza, a través de OpenRouter, el modelo `mistralai/mistral-nemo` para generar retroalimentación. Se emplea un prompt estandarizado y una temperatura baja para obtener respuestas más consistentes, con un formato compacto y una nota estimada alineada con el enunciado [13].

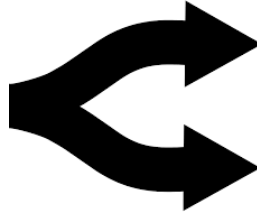


Figura 1.6. Logo de OpenRouter



Figura 1.7. Logo de Mistral

1.3.8 Bash

Bash es un intérprete de comandos muy extendido en sistemas Linux [6]. Se emplea para automatizar la integración con **VPL**: detectar archivos de código, leer ficheros de ejecución (enunciado, código base, casos), construir el JSON que se envía a la API y controlar ejecuciones duplicadas mediante guardas y bloqueos. Su uso permite adaptar el flujo sin modificar el comportamiento principal de VPL.



Figura 1.8. Logo de Bash

1.3.9 PuTTY

PuTTY es un cliente **SSH** para Windows que facilita el acceso remoto a servidores [19]. En este proyecto se utiliza para conectarse a una máquina de la universidad, en la cual vamos a poder desplegar la API, revisar registros, comprobar puertos abiertos y gestionar procesos. Esta combinación permite mantener el servicio activo y diagnosticar incidencias con rapidez.

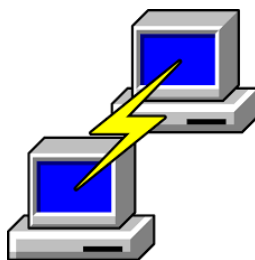


Figura 1.9. Logo de PuTTY

1.4 Metodología del trabajo

El desarrollo seguirá un enfoque **iterativo e incremental**, lo que permite ajustar el sistema de forma progresiva según los resultados de cada ciclo. Se empleará integración continua y pruebas en todas las etapas para optimizar el rendimiento y garantizar la estabilidad. Este enfoque facilita la detección temprana de errores, evita retrabajos costosos y permite integrar módulos de forma gradual, verificando su correcto funcionamiento antes de avanzar.

Cada ciclo de trabajo se organiza en **fases cortas** con el siguiente esquema:

- **Planificación breve.** Selección de tareas, criterios de aceptación y riesgos a cubrir en el ciclo (API, prompts, integración con VPL).
- **Implementación.** Desarrollo de la funcionalidad acordada (endpoints, *scripts* de VPL, ajustes de despliegue).
- **Pruebas.** Ejecución de pruebas unitarias y de integración, validación del endpoint */health*, pruebas de conectividad desde VPL y verificación del formato de la respuesta de */retroalimentacion*.
- **Revisión y mejora.** Análisis de resultados, refinamiento del *prompt* (longitud de líneas, tono, rúbrica de nota) y corrección de incidencias.
- **Documentación.** Actualización de la memoria, anexo de evidencias y guía de uso.

Para la **gestión del desarrollo** se emplearán ciclos cortos de implementación y prueba. Esto permite:

- **Validar requisitos** de forma continua y adaptar el diseño si es necesario.
- **Optimizar el análisis con LLM**, ajustando su comportamiento a partir de pruebas reales y evidencias de uso.
- **Garantizar compatibilidad con VPL**, manteniendo una integración fluida con el entorno del Campus Virtual y respetando sus restricciones.

Este enfoque busca un sistema **robusto, flexible y bien integrado**, minimizando riesgos y asegurando una evaluación automatizada eficaz para los ejercicios de programación, con capacidad de mejora continua en sucesivas iteraciones.

1.5 Estructura del documento

Este documento se estructura en siete capítulos y dos apéndices, además de la parte de referencias bibliográficas. El contenido de cada uno de ellos es el siguiente:

1. Introducción. Presenta la motivación, los objetivos, la metodología, la estructura del documento y un resumen de las tecnologías usadas, situando el papel de VPL y de los LLM en la retroalimentación del código.

2. Estado del arte y marco tecnológico. Describe VPL y el papel de los LLM en la retroalimentación, expone los límites (caja negra y no ejecución) y los criterios de selección. Justifica el marco tecnológico elegido y define contratos, despliegue y operación.

3. Análisis de requisitos. Define los requisitos funcionales y no funcionales, los criterios de evaluación de la retroalimentación, las restricciones del sistema (red, seguridad de credenciales) y los supuestos de uso.

4. Diseño de la arquitectura del sistema. Describe los componentes (VPL, API FastAPI, proveedor LLM), sus interfaces y contratos de datos, y la interacción entre ellos. Incluye consideraciones del despliegue y seguridad básica.

5. Implementación e integración. Detalla el desarrollo del módulo de consulta al LLM, sus pruebas, la integración con VPL mediante *scripts* y la validación de la integración en un entorno real.

6. Pruebas de retroalimentación y análisis. Evalúa la eficacia del sistema con ejercicios reales y distintos lenguajes, analiza las respuestas obtenidas y documenta mejoras en los prompts y en la integración.

7. Conclusiones y líneas futuras. Concluye la viabilidad de integrar VPL y un LLM vía API con retroalimentación compacto y operación reproducible, y propone ampliaciones: mejora de la ejecución, del *prompt* y de las salidas (en formato JSON), pasar a un LLM local, mayor optimización y personalización del sistema.

2

Estado del arte y marco tecnológico

2.1 Estado del arte

El estado del arte sitúa el proyecto en su contexto: se resumen las capacidades y límites de **VPL** como sistema de evaluación automática y el papel de los modelos de lenguaje de gran tamaño (**LLM**) en la generación de retroalimentación. Se revisan los enfoques existentes, los beneficios (objetividad, escalabilidad, claridad de la retroalimentación) y los riesgos (análisis estático, dependencia del contexto), con el fin de fundamentar la complementariedad entre las pruebas de caja negra y la retroalimentación pedagógica asistida por modelos de inteligencia artificial.

2.1.1 VPL

Virtual Programming Lab (VPL) es un módulo para Moodle orientado a gestionar prácticas de programación con edición en el navegador, ejecución interactiva y evaluación automática [22]. Su objetivo es facilitar el trabajo docente al ofrecer al estudiante un entorno integrado para codificar, probar y verificar sus soluciones sin salir del campus virtual. La ficha del plugin y el manual oficial describen su alcance funcional, con soporte para múltiples lenguajes, herramientas de detección de similitud y controles de edición.

Desde el punto de vista de la arquitectura, VPL combina tres piezas: el módulo de Moodle (actividad y gestión de envíos), el **VPL Jail System** (servidor de ejecución aislado) y un editor/IDE web integrado [21]. El *Jail System* actúa como un entorno de ejecución "sandbox" sin estado, capaz de compilar y ejecutar programas con límites de tiempo y memoria, manteniendo la seguridad y la separación respecto del sistema anfitrión.

El flujo típico de uso es el siguiente: el estudiante codifica y envía su solución en la actividad de Moodle; el módulo transfiere el código al *Jail System*; se compila y/o ejecuta con los ficheros de ejecución definidos por el docente; y, finalmente, se devuelven

al alumnado los resultados de ejecución y evaluación dentro de Moodle [22]. Esta separación entre interfaz y ejecución permite escalar la corrección en grupos numerosos y mantener una experiencia consistente.

La evaluación automática se basa en **BIOTES** (*Basic Input/Output Test Evaluation System*), un lenguaje de casos de prueba que permite describir entradas, salidas esperadas, límites de tiempo y otros parámetros [22]. El profesorado especifica los casos en el archivo «vpL_evaluate.cases» y VPL ejecuta el programa del estudiante comparando las salidas con lo esperado, produciendo un informe por caso y una calificación agregada. Además, VPL organiza la ejecución mediante ficheros como «run», «debug» y «evaluate», que orquestan compilación, pruebas y presentación de resultados, lo que da flexibilidad a la hora de ajustar el flujo según la asignatura.

En términos de enfoque, VPL ofrece principalmente evaluación de **caja negra**: la corrección depende de los casos de prueba definidos por el docente y de las restricciones de ejecución (tiempo, memoria y acceso a red) fijadas en el entorno aislado [22]. Este enfoque garantiza objetividad y repetibilidad, cualidades especialmente valiosas en asignaturas con gran número de estudiantes. A cambio, la retroalimentación textual resultante tiende a centrarse en si los casos pasan o fallan y en mensajes técnicos de ejecución, lo cual puede ser menos pedagógico cuando se busca orientar la mejora del código. Esta limitación ha impulsado la exploración de soluciones complementarias que aporten explicaciones y consejos más detallados sin comprometer la seguridad del entorno.

Por último, VPL mantiene documentación activa y versiones recientes tanto del plugin como del *Jail System*, con guías de instalación, actualización y administración. Esta base documental facilita su adopción institucional y su uso en asignaturas con distintos lenguajes, manteniendo coherencia en la experiencia docente y del alumnado [22].

2.1.2 LLMs en evaluación de código

Los modelos de lenguaje de gran tamaño (**LLM**) se emplean cada vez más para apoyar la enseñanza y la evaluación de programación por su capacidad para generar retroalimentación textual comprensible y orientada al aprendizaje [4]. Cuando se proporciona contexto suficiente —enunciado, código base, solución del alumno y, si existe, resultados de ejecución—, los LLM pueden explicar la intención del código, señalar errores comunes, sugerir mejoras y destacar buenas prácticas [2] [1]. En este sentido, actúan como un “tutor conversacional” capaz de adaptar el nivel de detalle de la explicación al enunciado y a la solución propuesta.

La literatura reciente y revisiones técnicas señalan que los LLM son útiles para la retroalimentación formativa, la detección de patrones de error y recomendaciones de estilo/estructura, con estudios en cursos reales y comparativas con docentes [2] [1]. También se destaca su capacidad para generar ejemplos adicionales, reformular enunciados y proponer pistas graduadas que ayudan al alumnado a avanzar sin revelar directamente la solución. Al mismo tiempo, presentan limitaciones relevantes: su análisis es estático si no se les aportan resultados de ejecución; pueden incurrir en respuestas imprecisas o directamente erróneas si el *prompt* es ambiguo o si el contexto es insuficiente; y su salida puede variar con pequeños cambios en el enunciado. Estas cuestiones se mitigan con formatos de salida controlados, reglas explícitas en el *prompt* y políticas de temperatura y longitud de respuesta adecuadas [9].

De forma más concreta, los trabajos revisados destacan varias capacidades de los LLM aplicadas a la corrección de código:

- **Detección y explicación de errores.** Pueden localizar errores sintácticos y lógicos habituales en ejercicios introductorios y de nivel intermedio e indicar en qué parte del código aparecen. En lugar de limitarse a señalar la línea problemática, son capaces de explicar por qué la construcción es incorrecta y qué efecto tiene en la ejecución [2] [1].
- **Generación de retroalimentación formativa.** Más allá de decidir si la solución es correcta, producen retroalimentación textual que contextualiza el fallo, relaciona el error con el enunciado y sugiere posibles estrategias de corrección. Esto permite que la retroalimentación sea más útil también cuando el alumno no ha superado la mayoría de los casos de prueba [2] [1].
- **Recomendaciones sobre estilo y estructura.** Pueden valorar aspectos como la claridad de los nombres, la modularización en funciones, la presencia de código duplicado o la complejidad innecesaria, proponiendo reorganizaciones que mejoran la legibilidad y la mantenibilidad incluso cuando el programa ya pasa las pruebas [9].
- **Uso combinado con resultados de ejecución.** Al incorporar salidas de pruebas automáticas, mensajes de error del compilador o trazas de ejecución, el modelo puede razonar no sólo sobre el código, sino también sobre su comportamiento observado. Esto permite diagnósticos más precisos que un análisis puramente estático, y se alinea bien con entornos que ya disponen de evaluación automática tipo caja negra [9].

En cuanto a las familias de modelos LLM, la literatura y la oferta actual de proveedores distinguen varias opciones reconocidas que se emplean en educación y evaluación de código: **Mistral**, **DeepSeek**, **Llama** (Meta) y **GPT** (OpenAI). Cada familia equilibra de modo distinto calidad, latencia, licencia, y facilidad de integración por API, y se distribuye en distintas variantes de tamaño y capacidad.

- **Mistral.** Presenta buen desempeño en tareas de razonamiento y programación, con modelos de pesos abiertos y oferta comercial por API; la documentación destaca versiones ajustadas a diferentes tamaños de contexto y requisitos de cómputo [11]. Algunas variantes se distribuyen bajo licencias específicas, lo que obliga a revisar las condiciones de uso en entornos docentes.



Figura 2.1. Logo de Mistral

- **DeepSeek.** Se orienta al razonamiento y a la eficiencia, con modelos recientes de buen rendimiento y acceso vía web, aplicación y API; la documentación resalta la relación entre eficiencia y calidad frente a otros modelos de tamaño similar



Figura 2.2. Logo de Deepseek

[5]. El ecosistema y las políticas están en evolución, por lo que se recomienda seguir de cerca cambios en catálogo, límites y condiciones de uso.

- **Llama** (Meta). Ofrece pesos abiertos, un ecosistema amplio y la posibilidad de integración vía pasarelas, con iteraciones recientes que incorporan mejoras sustanciales en la comprensión y generación de código [8]. A cambio, un despliegue local exige infraestructura propia y una lectura atenta de licencias y restricciones de uso, especialmente en contextos institucionales.



Figura 2.3. Logo de Llama (Meta)

- **GPT** (OpenAI). Aporta alto rendimiento general y mejoras continuas en codificación, seguimiento de instrucciones y manejo de contextos largos [14]. La contrapartida es un modelo propietario, con costes y condiciones de uso definidos por el proveedor, lo que puede influir en su adopción en entornos educativos con presupuestos limitados.



Figura 2.4. Logo de GPT

Además de los modelos generalistas, los proveedores han publicado variantes especializadas en programación dentro de estas mismas familias, diseñadas específicamente para la generación y el análisis de código en múltiples lenguajes. Estas

variantes se centran en tareas como completar funciones, refactorizar fragmentos, explicar algoritmos y sugerir pruebas adicionales, y la literatura las propone como opciones especialmente adecuadas para escenarios de tutorización y corrección de ejercicios prácticos en asignaturas de programación [11] [5][8] [14].

En conjunto, el estado del arte apunta a que la combinación de pruebas objetivas de caja negra con retroalimentación generada por LLM proporciona un equilibrio entre rigurosidad y valor pedagógico. Las pruebas garantizan corrección funcional y objetividad, mientras que el modelo aporta explicaciones y sugerencias adaptadas al contexto del ejercicio. Esta complementariedad justifica integrar VPL con un servicio intermedio basado en LLM, preservando el aislamiento del entorno de ejecución y estandarizando el formato de la retroalimentación para su consumo en el campus virtual [2] [1] [9].

2.2 Marco tecnológico

El marco tecnológico describe el *stack* adoptado y sus motivos: una API con **FastAPI** servida por **Uvicorn**, una pasarela **OpenRouter** y un modelo de la familia **Mistral**, integrados con **VPL** mediante *scripts* en *Bash*. Se detallan criterios de selección, decisiones de diseño y condicionantes operativos (versiones, variables en archivo de entorno, CORS, puertos, temporizadores), buscando reproducibilidad y una integración fiable en el entorno del campus.

2.2.1 Stack seleccionado

A partir de las tecnologías descritas en la sección *Tecnologías usadas*, el sistema se organiza en varios componentes que colaboran en el flujo completo de corrección, desde la entrega del código por parte del alumnado hasta la generación de la retroalimentación.

VPL/Moodle. Actúa como capa docente y punto de entrada del sistema: el alumnado entrega su código en actividades de VPL y el profesorado configura el *script* de evaluación asociado. VPL se encarga de ejecutar dicho *script* dentro del *Jail*, recoger la salida y mostrar la nota y la retroalimentación en la interfaz de Moodle, de modo que la integración con la herramienta habitual de las asignaturas sea transparente.

API con FastAPI. La API constituye la capa de servicio de IA. Expone el *endpoint* */retroalimentacion*, que recibe un JSON con el enunciado, el código base, el código del alumno, las restricciones y las evidencias de ejecución (*log* de *BIOTES* o resumen de casos del *fallback*). A partir de estos datos, construye el *prompt*, delega la llamada al modelo LLM y devuelve a VPL un texto de corrección estructurado junto con una nota global y campos auxiliares (*nota_tests*, *nota_ia*).

Servidor ASGI (Uvicorn). Uvicorn se utiliza como servidor ASGI ligero para publicar la API en un puerto accesible desde el servidor de Moodle. Su configuración se realiza mediante un archivo de entorno para la clave API y el comando de arranque, de forma que el despliegue en la máquina de la universidad se reduzca a activar el entorno virtual y lanzar un único proceso que atiende las peticiones procedentes de VPL.

Pasarela LLM (OpenRouter). La API no llama directamente a un proveedor concreto de modelos, sino que utiliza OpenRouter como intermediario. En el código solo se gestiona una clave API y un identificador de modelo, mientras que la selección del proveedor, la facturación y la rotación de modelos quedan encapsuladas en OpenRouter. Esto permite cambiar de modelo o de familia sin modificar el contrato entre VPL y la API.

Modelo LLM (Mistral). El identificador del modelo se fija en la configuración de la API (por ejemplo, *mistralai/mistral-nemo*) y puede modificarse en esta misma. El diseño del *prompt* y del formato de salida está pensado para ser compatible con distintos modelos, de modo que en trabajos futuros se puedan comparar variantes sin alterar el flujo de integración con VPL.

Scripts Bash. Dentro del *Jail* de VPL, el *script* *vpL_evaluate.sh* implementa la lógica de integración: detecta el archivo de código del alumno en distintos lenguajes, intenta lanzar *BIOTES* cuando están presentes *vpL_evaluate.cpp* y el fichero de casos, y, si no es posible, recurre a un *fallback* que ejecuta el programa caso a caso. En ambos escenarios, el *script* compone un JSON con toda la información relevante y lo envía a la API mediante *curl*.

PuTTY (acceso remoto). Desde los equipos Windows del entorno docente se emplea PuTTY para conectarse por SSH a la máquina de la universidad donde se despliega la API. A través de esta herramienta se realizan tareas como activar el entorno virtual, lanzar o reiniciar Uvicorn, consultar *logs* y comprobar conectividad, completando así el ciclo de operación del sistema.

CORS y seguridad básica. La API incorpora un *middleware* de CORS y una configuración mínima de cabeceras y métodos permitidos. Aunque en la situación actual solo se consume desde VPL, esta configuración deja preparado el servicio para posibles clientes web futuros (por ejemplo, paneles docentes) y evita problemas de navegador durante las pruebas sin introducir complejidad adicional en el *stack*.

2.2.2 Justificación de decisiones

Las decisiones tecnológicas se vinculan a requisitos del contexto docente y a las herramientas ya desplegadas en la universidad.

Integración con VPL/Moodle. VPL ya se utiliza en la UMA como herramienta oficial para la evaluación automática de prácticas de programación integrada en Moodle. Esto hace poco razonable adoptar jueces en línea externos o plugins alternativos cuando el objetivo del TFG es ampliar la funcionalidad existente y no sustituirla. Aprovechar VPL permite reutilizar el *Jail System* y mantener una experiencia homogénea para el alumnado.

Separación VPL-API. VPL se ejecuta en un *Jail* con restricciones de red y seguridad. Ejecutar modelos de lenguaje dentro del propio entorno de VPL o del servidor de Moodle resultaría más complejo y menos seguro. Por ello se opta por externalizar la lógica de generación de retroalimentación en una API independiente, a la que el *script* de evaluación envía únicamente el contexto necesario.

Elección de FastAPI. Frente a alternativas como Flask o Django REST Framework, se escoge FastAPI por su rapidez de desarrollo, el uso de anotaciones de tipo en Python y la generación automática de documentación, lo que facilita validar el contrato con VPL y depurar casos reales. Para el alcance de un TFG y un servicio de tamaño reducido, ofrece un equilibrio adecuado entre simplicidad y buenas prácticas de diseño de APIs.

Despliegue con Uvicorn. FastAPI está pensado para ejecutarse sobre servidores ASGI. Uvicorn proporciona un servidor ligero y sencillo de configurar, frente a otros servidores más pesados u orientados a WSGI. La opción de cargar variables desde un archivo de entorno (*--env-file*) facilita separar credenciales y parámetros del código y simplifica la rotación de claves como *OPENROUTER_API_KEY*.

Uso de OpenRouter. En lugar de integrar directamente la API de cada proveedor de modelos (por ejemplo, APIs específicas

de OpenAI o Grok) o desplegar un modelo propio en la infraestructura de la universidad, se recurre a OpenRouter como pasarela. La justificación principal es la facilidad de puesta en marcha: basta con una única cuenta y una clave de API para acceder a distintos modelos, sin tener que repetir procesos de alta, configuración y facturación en cada servicio. Además, OpenRouter ofrece un uso gratuito básico, lo que resulta especialmente adecuado para un TFG y para pruebas en entorno docente, mientras que la complejidad de costes y suscripciones queda abstraída fuera del sistema desarrollado.

Elección de la familia Mistral. A partir del estado del arte, se consideran varias familias de modelos (Mistral, DeepSeek, Llama, GPT) a nivel conceptual. Para este TFG se opta por la línea Mistral porque ofrece buen comportamiento en tareas de programación y razonamiento y se encuentra disponible en el catálogo de OpenRouter con una integración sencilla, escogiendo el modelo más potente de Mistral disponible en la pasarela, el «mistral-nemo».

Bash y curl en el Jail. VPL está diseñado para ejecutar *scripts* de *shell* como punto de entrada a la evaluación. Implementar la integración con la API en *Bash*, usando *curl* como cliente HTTP, evita introducir nuevos lenguajes o dependencias en el *Jail* y sigue el modelo habitual de funcionamiento del módulo. Otras opciones (como clientes en Python o Node.js dentro del *Jail*) aumentarían la complejidad de instalación.

Uso de PuTTY. En los equipos Windows del entorno docente se dispone de PuTTY como cliente SSH habitual. Aunque existen alternativas (como terminales integrados en IDE, WSL u otros clientes), el uso de PuTTY se ajusta a la infraestructura disponible, reduciendo la necesidad de introducir herramientas nuevas.

Configuración de CORS. Dejar la API sin configuración de CORS complicaría futuras integraciones con *front-end* web o paneles de administración que consuman directamente la API desde el navegador. La inclusión de un *middleware* de CORS desde el inicio permite controlar orígenes y métodos permitidos y evita errores de navegador en pruebas, sin afectar al uso actual desde VPL.

2.2.3 Condicionantes y versiones

Se documentan a continuación las condiciones técnicas de operación para reproducibilidad y auditoría:

Versionado y compatibilidad. Se fija Python 3.11 o superior con un entorno virtual aislado; FastAPI y Uvicorn en versiones estables recientes; y la librería requests como cliente HTTP. La documentación de FastAPI y Uvicorn se utiliza como guía de referencia para uso y despliegue.

Configuración por entorno. Las credenciales y parámetros se cargan desde un archivo *.env* (por ejemplo, OPENROUTER_API_KEY). Uvicorn permite la opción **—env-file** para inyectar el entorno antes de arrancar la aplicación. Se evita registrar las claves en *logs*.

Endpoints y contrato. La API expone */health* (comprobación rápida) y */retroalimentacion* (principal). La petición JSON incluye al menos enunciado, *codigo_base*, *codigo* y *casos_prueba*, y además:

- *restricciones* (o su alias *instrucciones*), con las restricciones estructuradas en Obligaciones y Recomendaciones.
- *resultados_casos* (o su alias *biotes_log*), que resume los resultados de los casos de prueba ejecutados en el *Jail*.

La respuesta devuelve la retroalimentación textual y una nota global en escala 0–10, junto con campos auxiliares:

- `nota_tests`, derivada de los casos de prueba.
- `nota_ia`, propuesta por el modelo.

Llamadas al proveedor LLM. Se usa el *endpoint* de chat *completions* con *messages* y se gestionan parámetros como *model*, *temperature* y *max_tokens*. Se soportan cabeceras opcionales de atribución (HTTP-Referer, APP-Title). Al cambiar de modelo (por ejemplo, dentro del catálogo Mistral), se mantiene el contrato de la API.

CORS y errores. Se configura el *middleware* de CORS para permitir orígenes concretos o un comodín durante pruebas. Es recomendable ubicar el *middleware* por fuera del árbol de rutas para que también se apliquen cabeceras CORS en respuestas de error.

Operación. Se recomienda ejecución con Uvicorn en segundo plano y comprobación del puerto (por ejemplo, con herramientas de sistema), consulta de *logs* y prueba de */health* tras cada despliegue. El acceso SSH desde Windows se realiza con PuTTY. Para tareas en *shell* y automatización de llamadas HTTP desde VPL se emplean *Bash* y *curl*.

Limitaciones prácticas. El tamaño de entrada (código y casos) y los timeouts deben mantenerse en márgenes razonables por latencia de red y límite de tokens. La documentación de Mistral señala que el catálogo se actualiza y pueden quedar obsoletos algunos modelos, por lo que conviene controlar el identificador exacto del modelo y planificar migraciones.

3

Análisis de requisitos

Este capítulo define qué debe hacer el sistema y en qué condiciones debe hacerlo, enumerando los requisitos funcionales (comportamiento esperado) y los requisitos no funcionales (propiedades de calidad, operación, seguridad y mantenimiento).

El sistema combina la ejecución automática de pruebas mediante **BIOTES**, el análisis del código por parte de un **modelo de lenguaje** y el uso opcional de **restricciones** docentes. Por ello, los requisitos especifican las entradas que recibe la **API** (enunciado, código, código base, casos de prueba, restricciones y resultados de **BIOTES**), el formato de la retroalimentación, el tratamiento de errores y la generación de notas parciales y finales.

La nota producida por el modelo es siempre estimativa y complementa, pero no debe de sustituir, la calificación oficial proporcionada por un **docente**.

3.1 Requisitos funcionales

- **RF-01 — Endpoint de salud.** La *API* expone */health* y devuelve un 200 con información mínima (estado y modelo) para comprobar conectividad desde *VPL*.
- **RF-02 — Endpoint principal.** La *API* expone el *endpoint /retroalimentacion (POST)*, que recibe un cuerpo *JSON* con: enunciado, código base, código del alumnado, casos de prueba, restricciones y resultados de ejecución procedentes de **BIOTES**. Los campos pueden estar vacíos; el sistema debe seguir operando e indicar en la respuesta los supuestos aplicados.
- **RF-03 — Formato de salida.** La respuesta incluye retroalimentación en lenguaje natural y una nota final (0–10). Cuando exista evidencia de ejecución, la respuesta incorporará también la nota de los casos de prueba y la nota derivada del análisis del modelo. El formato de salida será *JSON*.
- **RF-04 — Contenido pedagógico y técnico.** La retroalimentación debe: (a) resumir el estado general de la solución, (b) indicar si el código se ajusta al enunciado y al código base, (c) reflejar los resultados de los casos de prueba realizados mediante **BIOTES**, (d) señalar si se cumplen las **restricciones** del ejercicio, distinguiendo entre Obligaciones y Recomendaciones, (e) identificar errores o riesgos, (f) aportar consejos de mejora que no afecten a la nota y (g) ofrecer próximos pasos accionables.

- **RF-05 — Cálculo de notas.** El sistema genera dos notas independientes: una basada en los casos de prueba ejecutados mediante **BIOTES** y otra derivada del análisis del modelo. La nota final es una combinación de ambas. Si no existe evidencia de ejecución, solo se devolverá la nota procedente del modelo.
- **RF-06 — Integración con VPL.** Desde *VPL*, un *script Bash* ejecuta **BIOTES**, captura su salida y prepara un *JSON* con enunciado, código base, código del alumnado, casos de prueba, restricciones y resultados de **BIOTES**. Luego realiza un **POST** a la *API*. Si algún archivo no existe, su campo correspondiente se envía vacío.
- **RF-07 — Manejo de errores.** La *API* devuelve códigos 4xx ante errores de entrada y 5xx ante fallos del proveedor externo. En ambos casos, responde con un mensaje legible sin bloquear la ejecución de *VPL*.
- **RF-08 — Gestión de credencial.** La *API* carga *OPENROUTER_API_KEY* del entorno y valida su presencia en cada petición. Si falta, responde 500 con mensaje claro sin exponer información sensible.
- **RF-09 — Multilenguaje.** El sistema acepta código en Python, C/C++, Java y JavaScript como texto plano, sin dependencias específicas del lenguaje en el servidor.
- **RF-10 — CORS.** La *API* debe estar preparada para habilitar CORS en pruebas o si se desarrolla una interfaz web futura.
- **RF-11 — Robustez del cuerpo de petición.** Ante *JSON* inválido, la *API* responde 400 con mensaje claro e incluye un fragmento acotado del cuerpo recibido para diagnóstico, evitando mostrar datos sensibles.
- **RF-12 — Evidencia de ejecución.** El sistema debe proporcionar al modelo toda la evidencia disponible procedente de **BIOTES**. Si no existen resultados de ejecución, la *API* deberá indicarlo explícitamente y evitar generar conclusiones inventadas sobre el comportamiento del código.

3.2 Requisitos no funcionales

- **RNF-01 — Disponibilidad operativa.** La *API* debe responder siempre a las peticiones procedentes de *VPL*, incluso en caso de fallo del proveedor externo. Si no es posible generar retroalimentación o nota, la *API* devolverá un mensaje claro y estructurado que permita a *VPL* completar la ejecución sin bloquearse.
- **RNF-02 — Rendimiento razonable.** Se espera una latencia razonable en la generación de retroalimentación (segundos). Los *timeouts* deben evitar bloqueos en *VPL*.
- **RNF-03 — Seguridad de credenciales.** La clave del proveedor se gestiona mediante variables de entorno (archivo *.env*). No se registran secretos en los *logs*.
- **RNF-04 — Protección de datos y minimización de logs.** Los *logs* contienen solo metadatos técnicos (*endpoint*, estado, latencia, tamaño aproximado). No se registran código del alumno, casos, enunciados ni credenciales.
- **RNF-05 — Legibilidad de la retroalimentación.** La salida debe mantener líneas cortas, estructura clara y separación entre la nota procedente de los casos de prueba, la nota generada por el modelo y la nota final. La retroalimentación debe incluir únicamente información relevante y evitar saturar la interfaz de *VPL*.

- **RNF-06 — Mantenibilidad.** Los parámetros operativos (modelo, temperatura, tokens, *timeouts*) se fijan como constantes. La única variable de entorno es *OPENROUTER_API_KEY*. El *prompt* se versiona en el repositorio.
- **RNF-07 — Escalabilidad ligera.** La *API* debe atender varias peticiones simultáneas sin degradación notable.
- **RNF-08 — Portabilidad.** El despliegue está verificado en servidores Linux mediante *Python*, entorno virtual, *Uvicorn* y archivo *.env*. El sistema puede reproducirse en otros sistemas operativos que cuenten con soporte para estas herramientas, siempre que mantengan una configuración equivalente.
- **RNF-09 — Resiliencia de integración.** El sistema debe tolerar fallos parciales en la ejecución de **BIOTES** o en la comunicación con el proveedor externo. En ausencia de evidencia de ejecución, la *API* debe indicarlo en la respuesta y evitar generar conclusiones inventadas, garantizando que la actividad en *VPL* puede finalizar correctamente.
- **RNF-10 — Ética y uso responsable.** La nota generada por el modelo se presenta como estimación pedagógica, no debería ser una calificación oficial.
- **RNF-11 — Evolución del proveedor.** Se fija el identificador exacto del modelo y se prevé su sustitución si el proveedor lo modifica o retira.

4

Diseño de la arquitectura del sistema

Vamos a definir la **estructura** del sistema, sus **componentes** y la **interacción** entre ellos para integrar VPL con una API intermedia que consulta un LLM externo y devuelve retroalimentación compacta y pedagógica.

4.1 Visión general

El sistema amplía el flujo habitual de una actividad de programación en **VPL** sin modificar la forma en la que el alumnado interactúa con la plataforma. El estudiante sigue entrando en la actividad, editando su código y lanzando la ejecución desde el editor de VPL. La diferencia está en lo que ocurre dentro del entorno de evaluación: además de compilar y ejecutar el programa, se recoge información adicional y se envía a una **API** externa encargada de generar la retroalimentación.

Cuando el alumnado solicita la evaluación, VPL crea el entorno aislado (*jail*) y ejecuta el *script* `vpL_evaluate.sh`. Este *script* identifica el archivo de código del alumno y lee, si existen, los ficheros auxiliares de la actividad: enunciado, código base y restricciones. A continuación invoca a **BIOTES** a través del fichero auxiliar `vpL_evaluate.cpp`, que VPL compila automáticamente dentro del entorno y que actúa como puente con el evaluador de casos definidos en `vpL_evaluate.cases`, fichero que es **obligatorio** en la actividad. Con todos estos elementos el *script* construye un objeto **JSON** que incluye código, contexto del ejercicio, restricciones y evidencia de ejecución.

El **JSON** se envía mediante una petición *HTTP POST* al *endpoint* `/retroalimentacion` de la **API**, implementada con FastAPI y desplegada sobre Uvicorn en una máquina con IP pública. La API valida la petición, interpreta los resultados de **BIOTES**, calcula una nota asociada a los casos de prueba y prepara un mensaje estructurado que servirá como contexto para el **modelo de lenguaje**. Con la credencial almacenada en el archivo `.env` (`OPENROUTER_API_KEY`), la API llama al proveedor *OpenRouter*, que a su vez ejecuta un modelo de la familia Mistral para generar el texto de retroalimentación y una nota estimativa basada en el análisis del código.

Por último, la API combina la información recibida (enunciado, código, casos, restricciones y resultados de **BIOTES**) con la respuesta del modelo y devuelve a VPL un **JSON** de salida que contiene la retroalimentación en lenguaje natural, la nota

asociada a los tests, la nota procedente del modelo y la nota final resultante. VPL muestra este informe al estudiante en la propia interfaz de la actividad, de modo que la experiencia de uso apenas cambia: el alumnado sigue viendo el resultado de sus entregas en VPL, pero ahora con una explicación más rica y detallada del comportamiento de su código.

4.2 Componentes del sistema

El sistema se compone de varios elementos que cooperan para generar retroalimentación pedagógica basada tanto en la ejecución real del código como en el análisis de un **modelo de lenguaje**. Algunos componentes pertenecen al propio entorno **VPL**, otros han sido desarrollados específicamente para este trabajo, y otros forman parte de la infraestructura necesaria para desplegar la **API** y contactar con el proveedor externo.

Actividad VPL en Moodle. La actividad de VPL es el punto de entrada del sistema. El alumnado escribe y entrega su código desde el editor de la plataforma, exactamente igual que en una práctica convencional. Al lanzar la evaluación, Moodle envía los ficheros de la actividad al servidor de VPL, donde se ejecutan todos los *scripts* de corrección asociados (*vpL_evaluate.sh*, *vpL_evaluate.cpp*, *vpL_evaluate.cases* y los archivos auxiliares).

Jail System. El *jail* es un entorno de ejecución aislado en el que se corre el código del alumnado. Cada vez que un estudiante lanza la evaluación, VPL crea un contenedor temporal con su propia carpeta de trabajo, sus ficheros y una configuración controlada del sistema operativo. Dentro de ese contenedor se copian el código entregado, los ficheros de evaluación y los programas necesarios para compilar y ejecutar.

Este aislamiento cumple dos objetivos: por un lado, proteger el servidor frente a posibles errores o comportamientos no deseados del código del alumno; por otro, garantizar que todas las ejecuciones se realizan en condiciones homogéneas, sin depender de la configuración personal de cada ordenador. Desde el punto de vista del resto del sistema, el *jail* actúa como una «caja negra» que recibe ficheros, ejecuta las herramientas de corrección y devuelve sus resultados.

BIOTES. El motor *BIOTES* es el encargado de ejecutar los casos de prueba definidos por el profesorado y de comprobar si la salida del programa coincide con lo esperado. Para ello utiliza la información contenida en *vpL_evaluate.cases* y el ejecutable del alumno preparado dentro del *jail*. Su salida incluye datos como el número de pruebas realizadas, cuántas se han superado y, en algunos formatos, un resumen de los errores detectados. Esta información se convierte en la evidencia objetiva de ejecución que utilizará la API para calcular la nota de tests y para informar al modelo de lenguaje.

Archivo *vpL_evaluate.cpp*. El fichero *vpL_evaluate.cpp* actúa como adaptador entre el motor *BIOTES* y el resto del sistema. Se compila dentro del *jail* para generar un ejecutable específico que lee el fichero de casos de prueba, lanza el programa del alumno y recoge los resultados. El *script* *vpL_evaluate.sh* se apoya en este ejecutable para obtener la salida de *BIOTES* en un formato conocido.

Archivo *vpL_evaluate.cases*. El archivo *vpL_evaluate.cases* es un componente **obligatorio**. Contiene la definición de los casos de prueba con los que se evalúa el programa: datos de entrada, salidas esperadas y reglas de comparación. Sin este fichero, el sistema no podría medir objetivamente el comportamiento del código, por lo que la nota basada en tests quedaría vacía y la retroalimentación del modelo se limitaría a un análisis estático.

Archivos auxiliares del ejercicio. La actividad puede incluir varios ficheros adicionales que enriquecen el contexto de la corrección:

- *enunciado.txt*: describe el enunciado del ejercicio de forma textual.
- *codigo_base.txt*: contiene el código base proporcionado por el profesor, que el alumnado debe respetar o completar.
- *restricciones.txt*: recoge las restricciones docentes, separando entre Obligaciones (criterios que afectan a la nota) y Recomendaciones (buenas prácticas o sugerencias de mejora).

Si alguno de estos archivos no está presente, su contenido se considera vacío en la comunicación con la API, y el modelo de lenguaje debe explicitar esta ausencia en la retroalimentación.

Máquina servidor de la API. La API se despliega en una máquina con IP pública, de forma que el *jail* de VPL pueda enviarle peticiones HTTP. En esta máquina se instala Python, se crea un entorno virtual y se arranca el servidor con Uvicorn. La clave de acceso al proveedor de modelos se almacena en un archivo .env mediante la variable OPENROUTER_API_KEY, evitando incluirla directamente en el código fuente. Cambiando la dirección IP_PUBLICA en el script de VPL es posible apuntar a distintas máquinas servidoras si en el futuro se replican las instalaciones.

Script Bash vpl_evaluate.sh. El script *vpl_evaluate.sh* es el componente central de integración entre VPL, BIOTES y la API. Su papel es coordinar todo lo que ocurre dentro del *jail* y preparar la información que se enviará al modelo. A grandes rasgos, su funcionamiento se puede dividir en tres bloques:

- *Preparación de la ejecución.* El script localiza el archivo de código del alumno, infiere el comando de ejecución adecuado según la extensión (Python, C, C++, Java y JavaScript) y crea un pequeño lanzador *vpl_test* para que BIOTES pueda ejecutar siempre el programa de la misma manera.
- *Obtención de la evidencia.* A continuación comprueba la presencia de los ficheros de contexto (*vpl_evaluate.cases*, *enunciado.txt*, *codigo_base.txt*, *restricciones.txt*) y, si están disponibles, ejecuta *vpl_evaluate.cpp* para lanzar BIOTES y capturar sus resultados. Si por algún motivo BIOTES no genera salida válida, el script activa un mecanismo alternativo que interpreta el fichero de casos, ejecuta el programa del alumno en un bucle y construye un resumen de pruebas en formato JSON.
- *Construcción y envío del JSON.* El script reúne el código del alumno, el enunciado, el código base, las restricciones, los casos de prueba y los resultados de ejecución en un único objeto JSON. Este objeto se envía mediante una petición HTTP al endpoint */retroalimentacion* de la API, y la respuesta se muestra en la salida de VPL, junto con una nota desglosada.
- *Obtención de la retroalimentación.* El bash recoge la retroalimentación y las notas que devuelve la API. Acto seguido estructura la salida que se va a devolver a VPL con la retroalimentación, la nota de la IA, la nota basada en los casos de prueba y la nota media de estas dos.

De esta forma, *vpl_evaluate.sh* convierte una ejecución estándar de VPL en una llamada estructurada a la API, sin que el alumnado tenga que modificar su forma habitual de trabajar con la plataforma.

API en FastAPI. La API está desarrollada con *FastAPI* y expone dos *endpoints* principales: */health*, que permite comprobar que el servicio está disponible, y */retroalimentacion*, que procesa las peticiones reales de corrección. Su comportamiento también puede agruparse en varios bloques:

- *Recepción y validación de datos.* La API recibe el cuerpo *JSON* enviado desde el *script* de VPL, extrae los campos relevantes (código, enunciado, código base, casos de prueba, restricciones y resultados de *BIOTES*) y aplica validaciones básicas para detectar errores de formato o entradas vacías.
- *Tratamiento de la evidencia.* A partir de los resultados de *BIOTES*, la API intenta calcular una nota según la cantidad de tests ha pasado (por ejemplo, número de casos superados sobre el total) y genera un pequeño resumen de los casos que más adelante se insertará en la retroalimentación. Cuando la evidencia llega en formato texto, se utilizan expresiones regulares para localizar patrones habituales en los informes de *BIOTES*.
- *Construcción del prompt.* Con toda la información anterior se construye un mensaje de entrada para el modelo de lenguaje, que incluye el enunciado, el código del alumno, el código base, las restricciones y la evidencia de ejecución. También se añaden instrucciones claras sobre cómo debe estructurarse la respuesta y cómo distinguir entre Obligaciones y Recomendaciones.
- *Llamada al modelo y respuesta.* Finalmente, la API envía el *prompt* a través de *OpenRouter*, recibe la retroalimentación generada por el modelo, extrae la nota estimativa de IA y combina esta nota con la nota de tests. El resultado se devuelve a VPL en formato *JSON*, incluyendo el texto de retroalimentación, la nota final y las notas parciales.

La API actúa así como un traductor entre el mundo de la ejecución automática de pruebas y el mundo del lenguaje natural, asegurando que el modelo reciba un contexto completo y que la respuesta vuelva en un formato utilizable por la interfaz de VPL.

Modelo de lenguaje vía OpenRouter. El **modelo de lenguaje** se accede a través del servicio *OpenRouter*, que actúa como pasarela hacia modelos de diferentes familias. Desde el punto de vista del sistema, este componente es un «servicio de caja negra» que recibe un mensaje con el enunciado, el código, los resultados de ejecución y las restricciones, y devuelve un texto en lenguaje natural que explica qué funciona, qué no funciona, cómo se ajusta el código a lo pedido y qué aspectos pueden mejorarse.

4.3 Estructura y flujo del sistema

El sistema sigue un flujo en cadena que conecta la entrega del alumnado en **VPL** con la ejecución de **BIOTES**, la integración realizada mediante el *script* *vpL_evaluate.sh* y el procesamiento final llevado a cabo por la API. El objetivo es que el modelo de lenguaje disponga tanto del código del estudiante como de la evidencia obtenida durante la ejecución real de los casos de prueba, generando así una retroalimentación fundamentada en datos objetivos.

El proceso comienza cuando el alumnado evalúa su entrega en VPL. El sistema crea un entorno aislado (*jail*) donde compila y ejecuta el *script* **vpL_evaluate.sh**. Para corregir la solución entregada, este llama a **BIOTES** utilizando el fichero **vpL_evaluate.cpp**. Este motor (**BIOTES**) ejecuta los casos definidos en *vpL_evaluate.cases*, fichero obligatorio en cada actividad, y produce un informe con las pruebas superadas, las pruebas falladas y la salida producida por el programa del alumno.

Una vez *BIOTES* finaliza su evaluación interna, el script *vpl_evaluate.sh* recoge su salida, y con esto ya tiene todos los ficheros de contexto que necesita el LLM:

- el **código** del alumnado
- el **enunciado** del ejercicio (si existe)
- el **código base** proporcionado por la asignatura (si existe)
- el **casos de prueba** proporcionado por el profesor
- las **restricciones** definidas por el profesorado (Obligaciones y Recomendaciones)
- los **resultados de ejecución** generados por **BIOTES**

Con estos elementos, el script *vpl_evaluate.sh*, construye un objeto **JSON** que representa el estado completo de la evaluación: tanto la parte estática (enunciado, restricciones, código del alumno, código base, casos de prueba) como la parte dinámica (resultados reales de ejecución). Este *JSON* se envía mediante una petición *HTTP POST* al endpoint */retroalimentacion* de la **API**, alojada en una máquina con **IP pública** y desplegada sobre Uvicorn.

Al recibir la petición, la API valida la estructura de los datos y extrae la **nota sobre los casos de prueba** a partir de la salida de **BIOTES**. A continuación compone un mensaje estructurado para el **modelo de lenguaje**, incluyendo:

- el enunciado del ejercicio
- el código entregado
- el código base
- las restricciones y su clasificación
- los casos de prueba
- la salida entera de *BIOTES*
- los criterios que debe aplicar el modelo
- el formato exacto de la retroalimentación

La API envía este mensaje al proveedor **OpenRouter**, autenticándose mediante la **OPENROUTER_API_KEY**. El proveedor ejecuta un modelo de la familia **Mistral** y devuelve un texto con observaciones, explicación de errores, recomendaciones y una **nota estimativa** (nota de la *IA*) basada en los criterios recibidos. La API extrae esta nota, combina ambas calificaciones (nota de los casos y la nota de la inteligencia artificial) y genera un **JSON de salida** con:

- la retroalimentación en lenguaje natural
- la nota basada en los casos de prueba

- la nota generada por el modelo
- la nota final resultante de ambas fuentes

Finalmente, este **JSON** se le devuelve a nuestro *script* **vpl_evaluate.sh**, y muestra este informe en la interfaz de ejecución.

El estudiante ve en un único bloque:

- si su código cumple el enunciado
- si respeta el código base
- cuántos casos de prueba superó
- qué errores aparecieron en la ejecución
- si cumple las restricciones obligatorias
- sugerencias de mejora
- las distintas notas generadas

Este diseño garantiza que la retroalimentación sea coherente, reproducible y alineada con la evidencia real de ejecución, combinando la objetividad de **BIOTES** con la capacidad explicativa de un **modelo de lenguaje**. Todo esto está representado de forma gráfica en la siguiente figura [6.4](#).

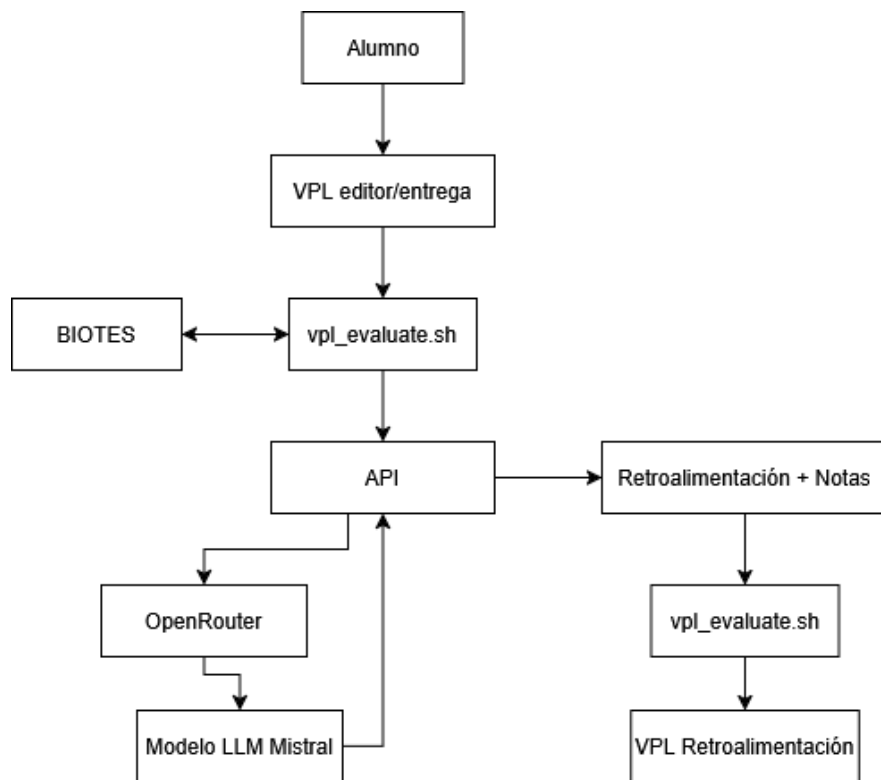


Figura 4.1. Estructura conceptual

5

Implementación e integración

En esta sección se describe la **implementación del sistema y su integración con VPL** entendida como un desarrollo conjunto de dos piezas principales: una API en FastAPI servida por *Uvicorn* y un *script* de evaluación en Bash que se ejecuta dentro del *Jail* de VPL. El *script* detecta el código entregado por el alumnado, ejecuta los casos de prueba mediante *BIOTES* o un mecanismo alternativo cuando el *tester* no está disponible y recopila el contexto necesario del ejercicio: enunciado, código base, restricciones y resultados de los casos.

A partir de esta información el *script* construye un objeto *JSON* y lo envía al *endpoint* *POST /retroalimentacion* de la API. La API consulta un modelo de la familia *Mistral* a través de *OpenRouter*, guiada por un mensaje de sistema y un *prompt* diseñados para generar retroalimentación compacta y una nota pedagógica. La respuesta incluye el texto de retroalimentación, la nota final y el desglose entre nota por tests y nota de la IA, que VPL muestra en la salida de la actividad. Además, la API expone un *endpoint* *GET /health* para comprobar su estado y gestiona la credencial *OPENROUTER_API_KEY* mediante un archivo de entorno *.env*, de modo que la configuración quede separada del código y sea reproducible en el servidor de la universidad.

A partir de esta arquitectura final, en los siguientes subapartados se detalla cómo fueron evolucionando conjuntamente la API y el *script* de VPL hasta llegar a esta versión estable.

5.1 Desarrollo conjunto de la API y del script de VPL

El desarrollo de este proyecto no se planteó como una API aislada a la que después se conectara VPL, sino como una **evolución conjunta** de dos piezas: por un lado, una API en FastAPI que consulta un modelo de lenguaje a través de *OpenRouter*, y por otro, un *script* de evaluación en Bash que se ejecuta dentro del *Jail* de VPL. En cada versión se modificaron tanto la API como el *script*, y se probaron juntos mediante actividades reales de VPL, ajustando el formato del *JSON*, el contenido del *prompt* y la forma de presentar el resultado al alumnado. A continuación se describen las principales iteraciones de este desarrollo conjunto.

5.1.1 Primera versión: prototipo local con LM Studio y envío de código

En la primera versión se construyó una **API mínima con FastAPI** destinada a probar de forma rápida la capacidad de un modelo de lenguaje ejecutado en local mediante la aplicación *LM Studio*. Se definió un único *endpoint POST* en */retroalimentacion* que recibía un *JSON* muy sencillo con un solo campo de texto. Ese campo contenía el código del alumno tal y como lo enviaba el cliente, y la API lo reenviaba directamente al modelo local con un mensaje de sistema que indicaba que debía actuar como profesor de programación. La salida del modelo se devolvía sin procesar, sin estructura de secciones ni nota numérica asociada.

En paralelo se creó un primer **script de evaluación** para VPL en Bash. Este *script* se ejecutaba dentro del *Jail* y buscaba el primer archivo con extensión *py* entregado por el estudiante en la actividad. Si no encontraba ningún archivo, mostraba un mensaje de error y asignaba directamente una nota cero. Si el archivo existía, leía todo el código, lo limpiaba de retornos de carro y construía a partir de él un *JSON* con un único campo de texto donde se incrustaba el código. Ese *JSON* se enviaba por *HTTP* a la API local mediante una petición *POST* y la respuesta se trataba como retroalimentación.

La retroalimentación recibida se mostraba en la salida de VPL como un comentario para el alumno, mientras que la nota se fijaba de forma fija a un valor máximo. En esta fase el objetivo no era todavía medir el rendimiento del código, sino comprobar que el flujo completo VPL → script Bash → API → modelo funcionaba y que el texto generado por el modelo podía verse en el panel de resultados de la actividad. Esta versión inicial estaba limitada a Python, no tenía en cuenta el enunciado ni los casos de prueba y tampoco distinguía entre nota por casos y nota de la inteligencia artificial, pero sirvió como prueba de concepto para validar la viabilidad de la integración.

5.1.2 Segunda versión: entrada estructurada y adaptación del script de VPL

En la segunda versión se mantuvo el uso de *LM Studio*, pero la API se refactorizó para trabajar con una **entrada más estructurada**. El *endpoint /retroalimentacion* dejó de recibir un único bloque de texto y pasó a aceptar un *JSON* con varios campos diferenciados: *enunciado*, *codigo_base*, *codigo* y *casos_prueba*. A partir de esos campos se construía un *prompt* enriquecido en el que se presentaban de forma ordenada la descripción del ejercicio, la plantilla inicial, la solución del alumno y una descripción de los casos de prueba. El modelo seguía trabajando en local, pero ahora disponía de un contexto mucho más completo para emitir una retroalimentación pedagógica sobre la solución, señalando si parecía cumplir o no el enunciado y comentando los casos descritos.

Para acompañar este cambio, el *script* de VPL dejó de enviar solo el código del alumno y se adaptó al nuevo formato de la API. Además de localizar el archivo de código y leer su contenido, el *script* empezaba a incorporar en el *JSON* otros **elementos relevantes** del ejercicio, como el enunciado y, cuando estaban disponibles, el código base y una descripción textual de los casos de prueba. La estructura enviada desde el *Jail* coincidía así con los campos que esperaba la API, lo que permitió probar por primera vez un flujo completo en el que el modelo veía no solo el código aislado, sino el contexto didáctico que lo rodeaba.

Con esta versión se realizaron las primeras pruebas de integración más realistas desde VPL. Se configuró una actividad de pruebas, se entregaron soluciones sencillas y se comprobó que el sistema generaba comentarios más coherentes con el enunciado y con la función esperada. Al intentar utilizar el modelo local desde el *Jail* se observó, sin embargo, una limitación importante: el entorno de ejecución de VPL no podía comunicarse con el servicio de *LM Studio* en la máquina del desarrollador, ya que se trataba de una dirección privada fuera del alcance del servidor de la universidad. Este resultado llevó a descartar el

despliegue del modelo en el equipo local y a plantear la necesidad de ejecutar la API en una máquina con dirección pública, capaz de recibir peticiones desde el *Jail* y desde otros entornos de prueba.

5.1.3 Tercera versión: proveedor Groq en servidor de la UMA

En la tercera versión se dio el salto desde un modelo local a un **proveedor externo de modelos**. La API pasó a ejecutarse en una máquina de la **universidad con dirección pública**, la cuál yo controlaba mediante la aplicación PuTTY ya explicada, y se configuró para llamar a través de la API de *Groq* (servicio en la nube orientado a la inferencia de modelos de lenguaje con baja latencia), a un modelo de la familia Llama. El *endpoint /retroalimentacion* seguía recibiendo un *JSON* estructurado con los campos *enunciado*, *codigo_base*, *codigo* y *casos_prueba*, pero ahora la petición se firmaba con una **clave de acceso gestionada como variable de entorno** en el servidor. Esta fue la primera versión en la que la credencial se externalizó explícitamente, dejando de estar fijada en el código fuente y permitiendo cambiarla o revocarla sin modificar la aplicación.

En esta etapa el *prompt* pedía al modelo que evaluara el código del alumno en función del enunciado, del código base y de los casos de prueba descritos, e incluía una instrucción para que añadiera al final una **nota numérica** en una escala de cero a cien. La redacción inicial sugería que el modelo debía comportarse como si estuviera ejecutando los casos de prueba sobre el código del alumno, aunque en realidad solo disponía de una descripción textual de ellos. Al revisar algunas respuestas se observó que el modelo afirmaba que ciertos casos pasaban o fallaban sin corresponderse exactamente con la ejecución real, lo que puso de manifiesto que no estaba ejecutando los casos, sino interpretando el texto del *prompt*.

A partir de estas observaciones se aclaró el papel de la nota generada por el modelo: en lugar de tratarla como un reflejo de la ejecución de casos de prueba, se comenzó a entender como una valoración estática basada en el cumplimiento del enunciado, el respeto al código base y la calidad general del código. La información sobre los casos de prueba seguía siendo útil como contexto, pero la responsabilidad de ejecutarlos y de obtener una nota objetiva por tests se reservó explícitamente a VPL y a BIOTES. Este cambio conceptual sirvió de base para las siguientes versiones, en las que se separan cada vez más la parte de evaluación automática de VPL y la parte de análisis pedagógico proporcionada por la inteligencia artificial.

El *script* de VPL evolucionó de forma moderada para adaptarse a este cambio de proveedor y a la ejecución en el servidor de la universidad. Se mantuvo la lógica de seleccionar el archivo de código del alumno, leer su contenido y construir un *JSON* con el enunciado, el código base y los casos de prueba, pero se actualizó la dirección de la API para apuntar a la nueva máquina con dirección pública y se ajustaron algunos mensajes de depuración. Con estas mejoras fue posible probar el flujo completo desde una actividad real de VPL: el alumno entregaba su solución, el *script* enviaba el contexto a la API en la máquina de la universidad y en la salida de la actividad aparecían la retroalimentación generada por el modelo y la nota numérica asociada, ahora interpretada como una valoración estática y no como resultado de ejecutar los casos.

5.1.4 Cuarta versión: migración a OpenRouter y reorganización del flujo en VPL

En la cuarta versión se sustituyó Groq por **OpenRouter** como pasarela de acceso a modelos de lenguaje. La API se reconfiguró para llamar al *endpoint* de *chat completions* de OpenRouter, utilizando un modelo de la familia Mistral y cargando la clave de acceso desde la variable de entorno *OPENROUTER_API_KEY*. Además se añadieron cabeceras opcionales de atribución para identificar la aplicación en el proveedor. En esta versión se incluyó por primera vez el *endpoint /health*, se creó como

comprobación rápida, devolviendo un indicador de estado y el nombre del modelo activo.

Hasta este momento toda la lógica de llamada al modelo se había ido incorporando directamente en el archivo *vpL_evaluate.sh*, que es el *script* de evaluación principal de VPL. Esta estrategia funcionaba para enviar el contexto a la API y mostrar la retroalimentación, pero tenía un efecto colateral importante: al convertir *vpL_evaluate.sh* en un *script* propio, se interfería con el flujo estándar de *BIOTES* y se dificultaba que VPL ejecutara los casos de prueba tal y como estaban definidos por el profesorado en *vpL_evaluate.cases*.

Para resolver este problema se revisaron los puntos de extensión que ofrece VPL y se optó por separar responsabilidades. En lugar de seguir sobrecargando *vpL_evaluate.sh*, la lógica de llamada a la API se movió a un *script* independiente denominado *pre_vpL_run.sh*, pensado para ejecutarse justo antes de la fase de evaluación. Durante las pruebas se experimentó con otros nombres de *script*, como *post_vpL_run.sh*, hasta comprobar que *pre_vpL_run.sh* permitía invocar al modelo al inicio de la evaluación sin impedir que a continuación VPL ejecutara *BIOTES* con normalidad.

El nuevo *pre_vpL_run.sh* comprueba primero que existe el archivo de casos *vpL_evaluate.cases* y aplica un mecanismo de bloqueo mediante un fichero temporal y un marcador en el directorio de trabajo para evitar ejecuciones duplicadas dentro del mismo *Jail*. Después localiza el archivo de código del alumno entre las extensiones habituales y reúne la información de contexto disponible: el enunciado del ejercicio (*enunciado.txt*), el código base (*codigo_base.txt*) y los propios casos de prueba. Todos estos elementos se leen de forma segura, permitiendo que alguno falte, y se ensamblan en un *JSON* con los campos *codigo*, *enunciado*, *codigo_base* y *casos_prueba*. El *script* realiza una consulta de salud al *endpoint* */health*, muestra un aviso si la respuesta no es correcta y, en cualquier caso, intenta enviar el *JSON* al *endpoint* */retroalimentacion*.

La respuesta de la API se guarda en un archivo auxiliar y se vuelca en la salida de VPL, mostrando tanto la retroalimentación textual como la nota que devuelve el modelo. El punto clave es que *pre_vpL_run.sh* termina siempre con un código de salida exitoso, de modo que VPL continúa ejecutando su flujo normal y *vpL_evaluate.sh* puede invocar *BIOTES* y procesar los casos definidos por el profesor. A partir de esta versión la integración queda claramente separada: el *script* previo se encarga de consultar al modelo de lenguaje y añadir un informe pedagógico, mientras que el evaluador estándar de VPL mantiene el control de la ejecución de los casos de prueba y de la nota objetiva asociada a ellos.

5.1.5 Quinta versión: integración final con *BIOTES*, restricciones y notas combinadas

En la quinta y última versión se consolida la **arquitectura actual del sistema**. Por el lado de VPL se vuelve a utilizar el fichero *vpL_evaluate.sh* como *script* principal de evaluación, y se introduce el fichero auxiliar *vpL_evaluate.cpp*. Este programa en C++ se compila dentro del *Jail* como *tester* y permite ejecutar los casos definidos en *vpL_evaluate.cases* al estilo de *BIOTES*. Si el *tester* oficial no llega a producir resultados aprovechables, el propio *script* activa un *fallback* multilenguaje que reproduce los casos y genera un resumen equivalente. En ambos escenarios se produce una salida estructurada en ficheros como *biotes_output.json* o *biotes_output.txt*, que servirá de evidencia objetiva para la API.

Además del código del alumno y del contexto mínimo (enunciado y código base), esta versión añade de forma estable nuevos ficheros de entrada: el texto de *restricciones.txt* (o, en su defecto, *instrucciones.txt*) y el resultado de *BIOTES* o del *tester* propio. El *script* *vpL_evaluate.sh* normaliza rutas y nombres alternativos (por ejemplo, variantes de código base), lee todos estos ficheros con un pequeño bloque en Python y construye un único objeto *JSON* con campos como *codigo*, *enunciado*, *codigo_base*,

casos_prueba, *restricciones* y **resultados_casos**. A partir de ahí realiza la llamada *POST* al *endpoint* */retroalimentacion* y, si la respuesta es correcta, muestra en la salida de VPL tanto la retroalimentación como las distintas notas recibidas (nota total y desglose).

La API también evoluciona de forma significativa en esta versión. En primer lugar incorpora funciones auxiliares para tratar la evidencia de ejecución: calcula una nota de tests entre 0 y 10, aceptando tanto *JSON* (con listas de casos aprobados o fallidos) como el *log* textual de *BIOTES*. A su vez, genera un pequeño resumen de casos que luego se inserta en la retroalimentación que ve el estudiante. Para evitar *prompts* excesivamente largos, se añaden límites de recorte y funciones de formateo que acotan el tamaño de cada bloque antes de enviarlo al modelo.

En segundo lugar se rediseña el mensaje de sistema y el *prompt* para el modelo de Mistral. El nuevo mensaje distingue explícitamente entre obligaciones y recomendaciones, pide que se evalúe cada restricción como *Cumple* o *No cumple*, y establece un formato de salida estructurado con secciones claras y un cierre obligatorio en la última línea con la forma *NOTA_IA: n – comentario*. La API limpia cualquier línea espuria, extrae esta nota de la IA, y la combina con la nota de tests calculada a partir de la evidencia: si existen ambas, devuelve la media redondeada; si solo hay una de ellas, utiliza esa; si no hay ninguna, indica que la nota no se ha podido proporcionar. La respuesta incluye, además de la retroalimentación completa, la nota final y el desglose en *nota_tests*, *nota_ia* y un breve comentario justificativo, que el *script* de VPL presenta en el cuadro de resultados.

Finalmente, se refuerza el **manejo de errores**: cuerpos *JSON* inválidos devuelven un código 400 con una muestra acotada del contenido; la ausencia de *OPENROUTER_API_KEY* genera un 500 claro; los errores de red producen un 502; y los códigos 4xx o 5xx de OpenRouter se propagan con el detalle en el campo de error correspondiente. Esto permite que *vpl_evaluate.sh* muestre mensajes comprensibles sin interrumpir la ejecución de los casos de prueba de VPL, completando así la integración entre *BIOTES* y la retroalimentación generada por el modelo de lenguaje.

5.1.6 Pruebas básicas en aislamiento

Antes de integrar la solución con VPL y con el *Jail*, conviene comprobar que la API funciona de forma aislada. Para ello se realizan un par de pruebas rápidas. El objetivo es verificar que el servicio arranca correctamente y que responde al *endpoint* de salud y al *endpoint* de retroalimentacion con consultas sencillas, sin implicar todavía al *script* de VPL ni a *BIOTES*.

5.1.6.1 PB1: comprobación de */health*

En esta prueba se lanza la API en local con Uvicorn, usando la clave de OpenRouter cargada desde el archivo de entorno, y en otra terminal se verifica que el *endpoint* de salud responde con un código 200 y un pequeño *JSON* con el estado y el modelo configurado.

- **Comando de consulta:**

```
curl -s http://150.214.214.19:8000/health
```

- **Resultado esperado:** respuesta *JSON* similar a: un campo *ok* con valor verdadero, el nombre del backend *openrouter* y el identificador del modelo activo.

- **Captura de ejemplo:** en la Figura 5.1 se muestra una salida típica en la que se observa que la API está viva y expone el modelo esperado.

```
C:\Users\manuv>curl http://127.0.0.1:8000/health
{"ok":true,"backend":"openrouter","model":"mistralai/mistral-nemo"}
```

Figura 5.1. Respuesta correcta del endpoint /health en local

5.1.6.2 PB2: prueba mínima de /retroalimentacion

En esta segunda prueba se comprueba el contrato básico del *endpoint* principal. Se envía manualmente un *JSON* muy sencillo con un enunciado corto y una solución correcta en Python, dejando el código base y los casos de prueba vacíos. El objetivo es verificar que la API acepta el cuerpo de la petición, construye el *prompt* para el modelo y devuelve una retroalimentación junto con una nota numérica a partir de ese contexto mínimo.

- **Comando de consulta:**

```
curl -s -X POST http://127.0.0.1:8000/retroalimentacion
-H "Content-Type: application/json"
--data-binary '{"enunciado":"Implementa suma(a,b).","codigo_base":"","codigo":"def suma(a,b): return a+b","casos_prueba":""}'
```

- **Resultado esperado:** respuesta *JSON* con al menos dos campos: *retroalimentacion*, que contiene el texto generado por el modelo siguiendo el formato pactado, y *nota*, que recoge la nota final calculada a partir de la información disponible.
- **Captura de ejemplo:** en la Figura 5.2 se observa un caso en el que la API devuelve una corrección breve para la función *suma* y una nota alta, al tratarse de una solución correcta y alineada con el enunciado.

```
C:\Users\manuv>curl -s -X POST http://127.0.0.1:8000/retroalimentacion -H "Content-Type: application/json" --data-binary '{"enunciado":"Implementa suma(a,b).","codigo_base":"","codigo":"def suma(a,b): return a+b","casos_prueba":""}'
{"retroalimentacion":"Corrección:\n- Resumen: La función suma(a, b) ha sido implementada correctamente.\n- Cumplimiento del enunciado: La función suma dos números y devuelve el resultado.\n- Ajuste al código base/contratos: No hay código base para comparar.\n- Complejidad/eficiencia y posibles errores: La función es eficiente y no hay errores obvios.\n- Análisis de casos: No hay evidencia de casos de prueba.\n- Chequeo de RESTRICCIONES OBLIGATORIAS: No hay restricciones obligatorias para verificar.\n- Consejos de mejora: Se podría agregar documentación a la función para explicar su propósito y parámetros.\n- Buenas prácticas detectadas: La función es simple y fácil de entender.\n- Próximos pasos: Se podrían agregar casos de prueba para validar la función.\n\nNOTA_IA: 9 - La función cumple con el enunciado y es eficiente, pero se podría mejorar con documentación y casos de prueba.\n\nNOTA_IA: 9 - La función cumple con el enunciado y es eficiente, pero se podría mejorar con documentación y casos de prueba.", "nota": "9", "nota_tests": "No disponible", "nota_ia": "9", "nota_ia_comentario": "La función cumple con el enunciado y es eficiente, pero se podría mejorar con documentación y casos de prueba."}
```

Figura 5.2. Prueba mínima del endpoint /retroalimentacion en local

5.2 Pruebas de la integración VPL-API

En este subapartado se comprueba que las funcionalidades principales de la integración funcionan de extremo a extremo dentro de VPL. El objetivo es verificar que el *script* de evaluación se ejecuta cuando corresponde, que construye el *JSON* con toda la información relevante, que la API calcula y devuelve las notas esperadas. La calidad pedagógica de la retroalimentación se analiza más adelante, en el apartado de experimentación.

5.2.1 PI1: Ejecución del tester y nota de tests

En esta prueba se verifica que el **tester** se ejecuta dentro del *Jail* y que su salida, cuando está disponible, se incorpora al *JSON* enviado a la API y permite calcular una **nota de tests** entre 0 y 10 a partir de la evidencia de ejecución. Para ello se utiliza una actividad de VPL que incluye el *script* de evaluación, el fichero *vpL_evaluate.cases* y el *tester* compilado. Al pulsar *Evaluate*, el propio *script* lanza el *tester* en el *Jail*, captura el *log* de ejecución y lo inserta en el campo de resultados del *JSON* que se envía a la API.

Observamos en la salida de VPL si se envían los resultados de *BIOTES*, si en la salida está la traza normal del *tester* y la retroalimentación generada por la API, donde aparece un bloque de resumen de casos insertado por el servidor y una nota de tests coherente con el número de pruebas superadas (Figura 5.3).

```
<|--  
-Summary of tests  
>+-----+  
>| 5 tests run/ 5 tests passed/ 0 tests skipped|  
>+-----+  
  
--|>  
  
Grade :=>>10  
  
NOTA_IA: 7 - Falta cumplir con las restricciones R3 y R4, y se podrían mejorar algunas prácticas.  
NOTA: 8  
NOTA_TESTS: 10
```

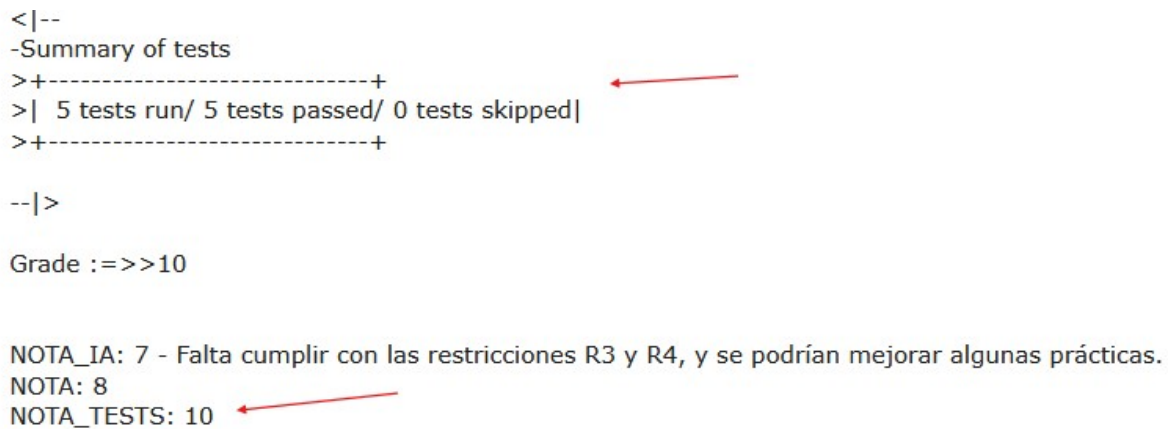


Figura 5.3. Resultado de *BIOTES* se envía a la API y se convierte en nota de tests

5.2.2 PI2: Construcción del JSON con contexto completo

En esta prueba se verifica que el *script* construye un **JSON** con el **contexto completo** del ejercicio y lo envía correctamente a la API. Para ello se configura una actividad que incluye los ficheros *enunciado*, *código base*, *casos de prueba*, *restricciones* y el código del alumno, y se lanza la evaluación desde VPL para revisar el contenido que el *script* envía a la API.

El objetivo es comprobar que el *JSON* contiene todos los campos previstos (*enunciado*, *código base*, *código del alumno*, *casos de prueba* y *restricciones*) y que, si falta alguno de los ficheros, ese campo se envía vacío sin romper la integración. La Figura 5.4 muestra un ejemplo de *JSON* enviado por el *script* con el contexto del ejercicio recogido de forma incompleta, falta el fichero de código base.

5.2.3 PI3: Integración independiente del lenguaje de programación

En esta prueba se verifica que la integración es **independiente del lenguaje de programación** y que la API trata siempre el código como texto, sin depender del lenguaje concreto. Para ello se reutiliza la misma integración, pero se cambian los lenguajes de los ejercicios (por ejemplo, Python, C++, C, Java y JavaScript); en cada caso se sube un archivo de código en el lenguaje correspondiente y se ejecuta la evaluación.

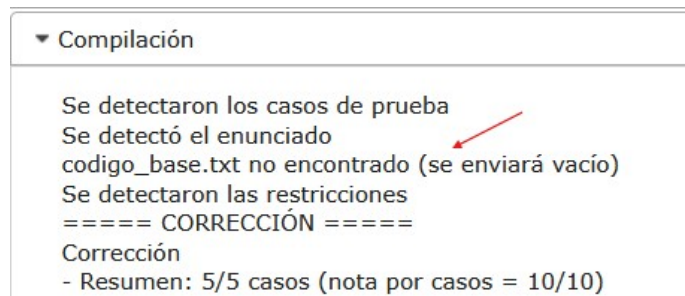


Figura 5.4. JSON enviado por el script con el contexto del ejercicio

El objetivo es comprobar que el *script* detecta el archivo de código adecuado por su extensión, construye el JSON sin errores y que la API responde con retroalimentación y nota en todos los casos, tratando el código siempre como texto. La Figura 5.5 recoge evaluaciones correctas en distintos lenguajes de programación.

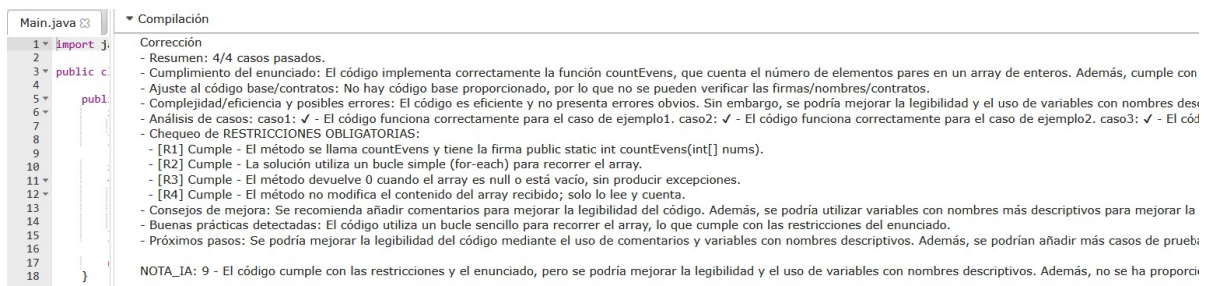


Figura 5.5. Evaluación correcta de un ejercicio Java

5.2.4 PI4: Cálculo y devolución de las notas

En esta prueba se verifica que la API calcula y devuelve correctamente el **desglose de notas** (nota por tests, nota de la IA y nota final) y que VPL lo muestra adecuadamente en la actividad. Para ello se ejecutan varias entregas sobre una misma actividad, algunas que superan todos los casos de prueba y otras que fallan una parte.

La evaluación permite observar los campos de nota que devuelve la API (*nota_tests*, *nota_ia* y *nota final*) y el modo en que se muestran en VPL. El objetivo es comprobar que la nota de tests se deriva de los resultados de *BIOTES*, que la nota de la IA se extrae del cierre de la respuesta del modelo y que la nota final coincide con la media entre ambas cuando las dos están disponibles. La Figura 5.6 muestra el desglose de nota de tests, nota de la IA y nota final en VPL.

Grade : =>>10

NOTA_IA: 7 - Falta cumplir con las restricciones R3 y R4, y se podrían mejorar algunas prácticas.
NOTA: 8
NOTA_TESTS: 10

Figura 5.6. Desglose de nota de tests, nota de la IA y nota final en VPL

5.2.5 PI5: tolerancia a fallos en la llamada a la API

En esta prueba se verifica el comportamiento de la integración cuando la llamada a la API falla y no es posible obtener retroalimentación ni notas. Se utiliza una actividad de VPL con el *script* de evaluación y el *tester* configurado. Al pulsar *Evaluate*, el *script* ejecuta el *tester* en el *Jail*, recoge su salida y construye el *JSON* con la evidencia, pero a continuación se fuerza un fallo en la llamada al modelo, por ejemplo deteniendo el servidor de la API o retirando temporalmente la clave de acceso.

El objetivo es comprobar que, en este escenario, la integración no se bloquea y muestra en VPL un **mensaje de error** claro procedente del *script*, en lugar de producir un fallo silencioso o una salida incoherente. En la salida de VPL no aparecen ni la retroalimentación ni las notas calculadas, ya que dependen completamente de la respuesta de la API, pero sí se ve el texto que indica que la petición ha fallado, con el código y el tipo de error devuelto. La Figura 5.7 ilustra este comportamiento, mostrando un ejemplo del mensaje que aparece cuando la API no está disponible.

```
Se detectaron los casos de prueba
Se detectó el enunciado
Se detectó el código base
Se detectaron las restricciones
curl: (7) Failed to connect to [REDACTED] port 8000 after 0 ms: Couldn't connect to server
===== CORRECCIÓN =====
⚠ Error al pedir feedback. HTTP=000000
```

Figura 5.7. Fallo en la API: mensaje de error mostrado en la salida de VPL

6

Pruebas de retroalimentación y análisis

En esta sección se evalúa el **comportamiento del sistema** cuando se aplica a ejercicios de programación reales. A priori no está claro en qué parte de la cadena pueden aparecer errores, o dónde pueden llegar a alucinar el LLM: ejecución de casos de prueba, construcción del *JSON*, evaluación de restricciones, cálculo de *nota_tests*, *nota_ia* o combinación final de ambas. El objetivo de este apartado es someter el sistema a muchas pruebas, recoger evidencias y, a partir de ellas, identificar patrones de comportamiento y posibles puntos débiles.

6.1 Metodología de las pruebas

Para estas pruebas se seleccionan varios ejercicios de programación con enunciado, código base, casos de prueba y restricciones definidos por el profesorado. Para cada ejercicio se preparan varias entregas representativas (por ejemplo, una solución correcta, una solución con errores en algunos casos y soluciones que incumplen algunas restricciones), de forma que se cubren distintos escenarios de corrección.

Todas las entregas se ejecutan en VPL utilizando la integración descrita previamente: el *script* de evaluación recoge el contexto del ejercicio, ejecuta los casos de prueba mediante *BIOTES* o *fallback* y envía el *JSON* resultante a la API, que devuelve retroalimentación, *nota_tests*, *nota_ia* y nota final. Paralelamente, el autor del trabajo realiza una corrección manual de referencia para cada entrega, que sirve como punto de comparación frente al comportamiento del sistema.

6.2 Resultados y evidencias

A partir de estas ejecuciones se observan varios patrones. En muchas entregas el sistema se comporta de forma coherente: la ejecución de casos de prueba se refleja en una *nota_tests* razonable, la retroalimentación describe con detalle los aciertos y errores del código y la *nota_ia* se sitúa en un rango similar al que elegiría un docente. En otras entregas, sin embargo, aparecen discrepancias puntuales entre la salida de la API y la corrección de referencia.

Estas discrepancias pueden, en principio, afectar a cualquiera de los componentes de la evaluación (tests, notas, restricciones, formato de la retroalimentación). Para concretar dónde suceden estos problemas, se analizan con más detalle cuatro casos representativos: uno en el que no se aprecian diferencias relevantes, y dos en los que sí que se observan divergencias entre la salida real y la esperada.

6.2.1 Estudio de caso 1: ejercicio sin discrepancias relevantes

En este primer caso vamos a utilizar este ejercicio:

- **Enunciado:** Queremos hacer un programa *a07e02.menor.py* que escriba el menor valor de una lista (se supone que la lista al menos tiene un valor). Para resolver el problema, cree una función llamada `def menor(l: list) -> int` que recibiendo una lista, nos diga cuál es el menor. El ejemplo anterior sería:

¿Cuántos valores tiene la lista? 3

Diga 3 valores: 5 -2 23

El menor de los elementos de la lista es -2

- **Código base:** `## Funciones`

```
def leer_lista() -> list:
```

```
## Programa principal
```

```
lista: list = leer_lista()
```

- **Restricciones:**

Obligaciones:

- * No usar nada que no hayamos visto (diccionarios...)
- * No usar funciones externas (de `math` ni otros módulos)
- * No se permiten listas por comprensión
- * No se permiten las funciones externas adicionales a `strip`, `append`, `len`, `in`, `input`, `print` y `round`
- * La E/S debe ser exactamente igual que en el enunciado
- * Use las sentencias de selección/repetición adecuadas para cada situación
- * Se usará el bucle `for` cuando sepamos el número de iteraciones antes de empezar el mismo
- * Se usará el bucle `while` cuando el número de iteraciones del mismo sea desconocido al inicio del mismo
- * No está permitido romper un bucle con `break` o `return`
- * En las funciones, solo habrá un `return` al final de las funciones (aunque es aceptable que si la función acaba con `if/else` cada uno

tenga un return en su interior)

Como este es el caso en el que no hay ningún problema con la salida, vamos a mostrar la retroalimentación de tres soluciones del ejercicio distintas:

- **1ª Solución:** Acierta todos los casos de prueba y cumple todas las restricciones. Este es el código del alumno que aporta como solución:

```
entrega.py
1 # Funciones
2 def leer_lista() -> list:
3     l: list = []
4     N: int = int(input("¿Cuántos valores tiene la lista? "))
5     print("Diga", N, "valores: ")
6     for i in range(N):
7         número: int = int(input())
8         l.append(número)
9     return l
10
11 def menor(l: list) -> int:
12     if len(l) == 0:
13         return 0
14     menor_valor = l[0]
15     for numero in l:
16         if numero < menor_valor:
17             menor_valor = numero
18     return menor_valor
19
20 # Programa principal
21 lista: list = leer_lista()
22
23 menor_valor = menor(lista)
24
25 print("El menor de los elementos de la lista es ", menor_valor)
```

Figura 6.1. Código de la solución 1

- **2ª Solución:** Acierta todos los casos de prueba, pero no cumple alguna Restricción. La restricción que falla, como podemos observar, es la de utilizar funciones externas. Esto se debe a que utiliza la función *min*. Este es el código del alumno que se aporta como solución:
- **3ª Solución:** Falla en un caso de prueba. Este es el código del alumno que aporta como solución:

Empezamos con la **primera solución**, la que acierta todos los casos de prueba y cumple con todo, esta nos da la siguiente retroalimentación:

===== CORRECCIÓN =====

Corrección

- Resumen: 10/10 casos pasados.
- Cumplimiento del enunciado: El código cumple con el enunciado de manera clara y precisa.
- Ajuste al código base: No hay código base proporcionado.
- Complejidad/eficiencia y posibles errores: El código es eficiente y no hay errores aparentes.

```

entrega.py
1 # Función
2 def leer_lista() -> list:
3     l: list = []
4     N: int = int(input("¿Cuántos valores tiene la lista? "))
5     print("Diga", N, "valores: ")
6     for i in range(N):
7         número: int = int(input())
8         l.append(número)
9     return l
10
11 # Leer menor valor en la lista
12 def menor(l: list) -> int:
13     return min(l)
14
15 # Programa principal
16 lista: list = leer_lista()
17
18 valor_menor = menor(lista)
19
20 print("El menor de los elementos de la lista es", valor_menor)

```

Figura 6.2. Código de la solución 2

- Análisis de casos: caso1: ☐ - caso2: ☐ - caso3: ☐ - caso4: ☐ - caso5: ☐ - caso6: ☐ - caso7: ☐ - caso8: ☐ - caso9: ☐ - caso10: ☐
- Chequeo de RESTRICCIONES OBLIGATORIAS:
 - [R1] Cumple: No se utilizan diccionarios ni otras estructuras no vistas.
 - [R2] Cumple: No se utilizan funciones externas de otros módulos.
 - [R3] Cumple: No se utilizan listas por comprensión.
 - [R4] Cumple: Solo se utilizan las funciones permitidas.
 - [R5] Cumple: La E/S es exactamente igual que en el enunciado.
 - [R6] Cumple: Se utilizan las sentencias de selección/repeticón adecuadas.
 - [R7] Cumple: Se utiliza el bucle for cuando se sabe el número de iteraciones.
 - [R8] Cumple: Se utiliza el bucle while cuando el número de iteraciones es desconocido.
 - [R9] Cumple: No se rompe un bucle con break o return.
 - [R10] Cumple: Solo hay un return al final de las funciones.
- Consejos de mejora: Se podrían utilizar constantes para mejorar la legibilidad del código.
- Buenas prácticas detectadas: El uso de comentarios y la indicación de los tipos de las variables son buenas prácticas.
- Próximos pasos: Se podría considerar el uso de funciones adicionales para mejorar la modularidad del código.

NOTA_IA: 9 - El código cumple con las restricciones obligatorias y se ajusta al enunciado de manera clara y precisa. Se podría mejorar la legibilidad del código con el uso de constantes.

— Resumen de casos (servidor) —

Testing 1/10 : ejemplo1

Testing 2/10 : ejemplo2

Testing 3/10 : ejemplo3

Testing 4/10 : ejemplo4

Testing 5/10 : ejemplo5

```

entrega.py
1 # Funciones
2 def leer_lista() -> list:
3     l: list = []
4     N: int = int(input("¿Cuántos valores tiene la lista? "))
5     print("Diga", N, "valores: ")
6     for i in range(N):
7         número: int = int(input())
8         l.append(número)
9     return l
10
11 def menor(l: list) -> int:
12     for i in range(len(l)):
13         menor = l[0]
14         if l[i] < menor:
15             menor = l[i]
16     return menor
17
18 # Programa principal
19 lista: list = leer_lista()
20 x: int = menor(lista)
21
22
23 print("El menor de los elementos de la lista es ", x)

```

Figura 6.3. Captura de la solución 3

Testing 6/10 : ejemplo6

Testing 7/10 : ejemplo7

Testing 8/10 : ejemplo8

Testing 9/10 : ejemplo9

Testing 10/10 : ejemplo10

<|-

-Summary of tests

>+-----+

>| 10 tests run/10 tests passed/ 0 tests skipped|

>+-----+

-|>

Grade :=>10

NOTA_IA: 9 - El código cumple con las restricciones obligatorias y se ajusta al enunciado de manera clara y precisa. Se podría mejorar la legibilidad del código con el uso de constantes.

NOTA_TESTS: 10

NOTA: 10

Continuamos con la **segunda solución** del ejercicio, la que acierta todos los casos de prueba, pero no cumple algunas restricciones, esta nos da la siguiente retroalimentación:

===== CORRECCIÓN =====

Corrección

- Resumen: 10/10 casos.
- Cumplimiento del enunciado: El código cumple con el enunciado de encontrar el menor valor de la lista.
- Ajuste al código base/contratos: No hay código base para comparar.
- Complejidad/eficiencia y posibles errores: El algoritmo es eficiente, ya que recorre la lista una sola vez. Sin embargo, se podría mejorar la legibilidad del código.
- Análisis de casos: Todos los casos pasan.
- Chequeo de RESTRICCIONES OBLIGATORIAS:
 - [R1] Cumple: No se han utilizado listas ni diccionarios.
 - [R2] No cumple: Se ha utilizado la función externa min.
 - [R3] Cumple: No se han utilizado listas por comprensión.
 - [R4] No cumple: Se ha utilizado la función externa min.
 - [R5] Cumple: La E/S es exactamente igual que en el enunciado.
 - [R6] Cumple: Se han utilizado las sentencias de selección/repeticón adecuadas.
 - [R7] Cumple: Se ha utilizado el bucle for cuando se sabía el número de iteraciones.
 - [R8] Cumple: No se ha utilizado el bucle while.
 - [R9] Cumple: No se ha utilizado break o return para romper un bucle.
 - [R10] Cumple: Solo hay un return al final de las funciones.
- Consejos de mejora: Se podría mejorar la legibilidad del código, utilizando comentarios y nombres de variables más descriptivos. También se podría utilizar una constante para almacenar el valor inicial del menor.
- Buenas prácticas detectadas: Se han utilizado identificadores significativos y se han indicado los tipos de las variables.
- Próximos pasos: Se podría mejorar la legibilidad del código y se podrían agregar más casos de prueba para asegurarse de que el código funciona correctamente en diferentes situaciones.

NOTA_IA: 7 - El código cumple con el enunciado y las restricciones obligatorias, pero se podría mejorar la legibilidad del código y se ha encontrado una violación de las restricciones R2 y R4.

— Resumen de casos (servidor) —

Testing 1/10 : ejemplo1

Testing 2/10 : ejemplo2

Testing 3/10 : ejemplo3

Testing 4/10 : ejemplo4

Testing 5/10 : ejemplo5

Testing 6/10 : ejemplo6

Testing 7/10 : ejemplo7

Testing 8/10 : ejemplo8

Testing 9/10 : ejemplo9

Testing 10/10 : ejemplo10

<|-

-Summary of tests

>+-----+

>| 10 tests run/10 tests passed/ 0 tests skipped|

>+-----+

-|>

Grade :=>10

NOTA_IA: 7 - El código cumple con el enunciado y las restricciones obligatorias, pero se podría mejorar la legibilidad del código y se ha encontrado una violación de las restricciones R2 y R4.

NOTA_TESTS: 10

NOTA: 9

Por último, comprobamos la retroalimentación que nos devuelve la solución que **no supera** todos los **casos de prueba**, es decir la solución 3:

===== CORRECCIÓN =====

Corrección

- Resumen: 9/10 casos pasados.

- Cumplimiento del enunciado: El código cumple con el enunciado, ya que lee una lista de números y muestra el menor valor de la lista.

- Ajuste al código base/contratos: No hay código base/contratos para comparar.

- Complejidad/eficiencia y posibles errores: El código es eficiente, ya que recorre la lista una sola vez para encontrar el menor valor. Sin embargo, hay un posible error en la función 'menor', ya que no se actualiza la variable 'menor' si el primer valor de la lista es mayor que el resto.

- Análisis de casos: caso1: ■ - El menor valor de la lista es -2. caso2: ■ - El menor valor de la lista es 8. caso3: ■ - El menor valor de la lista es -20. caso4: ■ - El menor valor de la lista es 1. caso5: ■ - El menor valor de la lista es 0. caso6: ■ - El menor valor de la lista es 100. caso7: ■ - El menor valor de la lista es -1. caso8: ■ - El menor valor de la lista es 1. caso9: ■ - El menor valor de la lista es -6. caso10: ■ - El menor valor de la lista es -2.

- Chequeo de RESTRICCIONES OBLIGATORIAS:

- [R1] Cumple - No se han utilizado listas ni diccionarios.

- [R2] Cumple - No se han utilizado funciones externas.

- [R3] Cumple - No se han utilizado listas por comprensión.

- [R4] Cumple - No se han utilizado funciones externas adicionales a 'input', 'print' y 'round'.

- [R5] Cumple - La entrada y salida son exactamente iguales que en el enunciado.

- [R6] Cumple - Se han utilizado las sentencias de selección/repetición adecuadas para cada situación.

- [R7] Cumple - Se ha utilizado el bucle 'for' para recorrer la lista.

- [R8] No aplica - No se ha utilizado el bucle 'while'.

- [R9] No aplica - No se ha utilizado 'break' ni 'return' para romper un bucle.

- [R10] No cumple - En la función 'menor', hay un 'return' dentro del bucle 'for'.
- Consejos de mejora: Se podría utilizar una variable 'menor' para almacenar el menor valor encontrado en la lista, en lugar de comparar cada valor con el primer valor de la lista. También se podrían agregar comentarios para explicar el funcionamiento del código.
- Buenas prácticas detectadas: Se han utilizado identificadores significativos y se han indicado los tipos de las variables.
- Próximos pasos: Se podría agregar una validación para asegurarse de que la lista no está vacía antes de buscar el menor valor. También se podría agregar una opción para salir del programa.

— Resumen de casos (servidor) —

Testing 1/10 : ejemplo1

Testing 2/10 : ejemplo2

Testing 3/10 : ejemplo3

Testing 4/10 : ejemplo4

Testing 5/10 : ejemplo5

Testing 6/10 : ejemplo6

Testing 7/10 : ejemplo7

Testing 8/10 : ejemplo8

Testing 9/10 : ejemplo9

Testing 10/10 : ejemplo10

<|-

-Test 1: ejemplo1 (-1.000)

Incorrect program output

— Input —

>3

>5

>-2

>23

— Program output —

>¿Cuántos valores tiene la lista? Diga 3 valores:

>El menor de los elementos de la lista es 5

— Expected output (text)—

>¿Cuántos valores tiene la lista?

>Diga 3 valores:

>El menor de los elementos de la lista es -2

-|>

<|-

-Summary of tests

>+-----+

>| 10 tests run/ 9 tests passed/ 0 tests skipped|

>+-----+

-|>

Grade :=» 9

NOTA_TESTS: 7

NOTA_TESTS: 9

NOTA: 8

6.2.2 Estudio de caso 2: ejercicio con problemas en la corrección de las restricciones

En el segundo caso vamos a utilizar este otro ejercicio:

- **Enunciado:** Queremos hacer un programa `a06e03.simplificar.py` que simplifique una fracción. Para simplificar una fracción debe dividir tanto el numerador y denominador entre su máximo común divisor. Para resolver el problema, primero cree una función llamada `mcd` que calcule el máximo común divisor de dos valores que recibe como parámetros y lo devuelva. Luego desarrolle una función, `escribir_simplificada`, que reciba dos valores, numerador y denominador, calcule su simplificada (usando una llamada a la función `mcd`) y luego escriba por pantalla la fracción simplificada (la función no debe devolver nada). Observe en los ejemplos que si el denominador al simplificarse es 1 no se escribirá. El programa principal deberá leer el numerador y denominador y luego escribir la fracción simplificada (usando la función previa) tal como se muestra en los próximos ejemplos:

Numerador: 20 Denominador: 70 La fracción simplificada es 2/7

Numerador: 30 Denominador: 12 La fracción simplificada es 5/2

Numerador: 42 Denominador: 21 La fracción simplificada es 2

- **Restricciones:**

Obligaciones:

- * No usar nada que no hayamos visto (listas, diccionarios...)
- * No usar funciones externas (de `math` ni otros módulos)
- * No se permiten listas por comprensión
- * No se permiten las funciones externas adicionales a `input`, `print` y `round`
- * La E/S debe ser exactamente igual que en el enunciado
- * Use las sentencias de selección/repetición adecuadas para cada situación
- * Se usará el bucle `for` cuando sepamos el número de iteraciones antes de empezar el mismo
- * Se usará el bucle `while` cuando el número de iteraciones del mismo sea desconocido al inicio del mismo

* No está permitido romper un bucle con break o return

* En las funciones, solo habrá un return al final de las funciones (aunque es aceptable que si la función acaba con if/else cada uno tenga un return en su interior)

- **Solución utilizada:** La solución que vamos a utilizar no incumplene ninguna restricción de las que hay. El código que genera esta retroalimentación es el siguiente:

```
entrega.py
1 # MCD de dos números
2 def mcd(a: int, b: int) -> int:
3     while b != 0:
4         a, b = b, a % b
5     return (a)
6
7 # Numerador y denominador, simplifica la fracción
8 def escribir_simplificada(numerador: int, denominador: int) -> None:
9     divisor = mcd(numerador, denominador)
10    num_simplificado = numerador // divisor
11    den_simplificado = denominador // divisor
12
13    # Mostrar la fracción simplificada, sin el denominador si es 1
14    if den_simplificado == 1:
15        print(f"La fracción simplificada es {num_simplificado}")
16    else:
17        print(f"La fracción simplificada es {num_simplificado}/{den_simplificado}")
18
19
20 # Programa principal
21 numerador = int(input("Numerador: "))
22 denominador = int(input("Denominador: "))
23
24 escribir_simplificada(numerador, denominador)
```

Figura 6.4. Captura de la solución correcta

Con estos campos que le mandamos al LLM recibimos esta retroalimentación devuelta:

===== CORRECCIÓN =====

Corrección

- Resumen: 10/10 casos.
- Cumplimiento del enunciado: El código del alumno cumple con el enunciado, ya que calcula el máximo común divisor de dos valores y simplifica una fracción.
- Ajuste al código base: No hay código base para comparar.
- Complejidad/eficiencia y posibles errores: El algoritmo de cálculo del MCD es eficiente y no hay errores obvios en el código.
- Análisis de casos: caso1: [] - caso2: [] - caso3: [] - caso4: [] - caso5: [] - caso6: [] - caso7: [] - caso8: [] - caso9: [] - caso10: []
- Chequeo de RESTRICCIONES OBLIGATORIAS:
- [R1] Cumple.
- [R2] Cumple.
- [R3] Cumple.
- [R4] Cumple.
- [R5] Cumple.

- [R6] No cumple. El bucle while se utiliza en la función mcd, pero no se indica que el número de iteraciones del mismo sea desconocido al inicio del mismo.
- [R7] No cumple. El bucle for no se utiliza en ningún caso.
- [R8] No cumple. El bucle while se utiliza en la función mcd, pero no se indica que el número de iteraciones del mismo sea desconocido al inicio del mismo.
- [R9] Cumple.
- [R10] Cumple.
- Consejos de mejora: Utilizar el bucle for en lugar del bucle while en la función mcd. Utilizar comentarios para explicar el código.
- Buenas prácticas detectadas: El uso de la función mcd para calcular el máximo común divisor es una buena práctica.
- Próximos pasos: Implementar el bucle for en la función mcd y agregar comentarios para mejorar la legibilidad del código.

— Resumen de casos (servidor) —

Testing 1/10 : ejemplo1

Testing 2/10 : ejemplo2

Testing 3/10 : ejemplo3

Testing 4/10 : ejemplo4

Testing 5/10 : ejemplo5

Testing 6/10 : ejemplo6

Testing 7/10 : ejemplo7

Testing 8/10 : ejemplo8

Testing 9/10 : ejemplo9

Testing 10/10 : ejemplo10

<|-

-Summary of tests

>+-----+

>| 10 tests run/10 tests passed/ 0 tests skipped|

>+-----+

-|>

Grade :=>10

NOTA_IA: 8 - El código del alumno cumple con la mayoría de las restricciones y el enunciado, pero no utiliza el bucle for en la función mcd y no hay comentarios en el código.

NOTA_TESTS: 10

NOTA: 9

Estos ejemplos representan lo que se ha encontrado en el período de pruebas del sistema. Los ejercicios utilizados pertenecen

a distintas asignaturas del Grado en Ingeniería Informática y cubren niveles y estilos de enunciado variados, desde problemas introductorios hasta prácticas de cursos avanzados. A partir de estas evidencias, en el siguiente apartado se analizan con más detalle las pruebas y sus implicaciones para el uso del sistema en contexto docente.

6.3 Análisis de los errores observados

A partir de los estudios de caso anteriores y de otras ejecuciones realizadas, se observa que la mayoría de las correcciones se comportan de manera coherente: la *nota_tests* refleja fielmente los resultados de *BIOTES* o del *fallback*, la retroalimentación describe de forma razonable los aciertos y errores del código y la *nota_ia* suele situarse en un rango compatible con la percepción de un docente. Los problemas aparecen en un subconjunto de entregas y se concentran en tres tipos de situaciones: la **evaluación de restricciones**, la **sensibilidad a cómo se formatea el contexto** (enunciado, código base y restricciones) y, de forma más esporádica, el **formato de la salida generada por el modelo**.

En primer lugar, los errores más visibles se producen en el chequeo de **restricciones**. En ejercicios como el del apartado anterior se aprecia que el sistema marca algunas obligaciones como *Cumple* cuando en realidad la solución incumple algún detalle (por ejemplo, la igualdad exacta entre la E/S y el enunciado) o, en otros casos, podría marcar *No cumple* sin aportar una evidencia suficiente en el código. Este comportamiento no se debe a una restricción concreta, sino a la combinación de dos factores: por un lado, ciertas restricciones están redactadas de forma larga o agrupan varias ideas en la misma frase, lo que complica que el modelo las verifique de manera mecánica; por otro lado, el LLM tiende a interpretar el texto con cierto margen de inferencia y puede alucinar parcialmente al decidir si algo se cumple o no. En consecuencia, el chequeo detallado de **restricciones largas o semánticamente complejas** resulta menos fiable que la evaluación global del ejercicio.

En segundo lugar, se aprecia que el sistema es sensible a la **claridad y estructura** de los campos de entrada que se envían a la API. Cuando el enunciado, el código base y las restricciones están bien delimitados y presentan una estructura esperable, la retroalimentación hace un uso correcto de ellos. Sin embargo, si el **código base es demasiado escueto** o se parece más a un esqueleto minimalista, el modelo puede llegar a afirmar que “no hay código base para comparar” aunque sí exista contenido. En la misma línea de lo comentado en el párrafo anterior, cuando una restricción está formulada de forma ambigua, demasiado larga o mezclando varias ideas, el modelo puede interpretarla solo de manera parcial y etiquetar *Cumple* o *No cumple* de forma discutible, aunque el resto del contexto sea correcto.

Por último, durante el periodo de pruebas también se observan, aunque con una **frecuencia muy baja**, **errores en la estructura** de la retroalimentación generada por el modelo. En algunos casos aislados el LLM devuelve el contenido en un bloque de texto más compacto de lo esperado, sin respetar exactamente el formato de líneas establecido, o bien altera ligeramente la línea final de *NOTA_IA* de forma que resulta más difícil extraer la nota automáticamente, llegando incluso al caso de que no enseñe esta nota. La API incorpora ciertas defensas (por ejemplo, búsqueda robusta de la última línea que comienza por *NOTA_IA*: y purga de campos no deseados), pero no puede garantizar que el formato se cumpla siempre al 100%. En la práctica, cuando aparece este tipo de error, suele bastar con volver a evaluar la entrega para obtener una salida que sí respete la estructura acordada.

En conjunto, el análisis de los errores observados sugiere que el sistema es más fiable cuando trabaja con señales **muy objetivas** (resultados de casos de prueba, presencia de ciertos patrones sintácticos) y muestra mayor variabilidad cuando debe

interpretar restricciones largas o mantener un formato de salida muy rígido. Estas observaciones refuerzan la idea de que la retroalimentación automática puede servir como primera guía para el estudiante, pero requiere una redacción cuidadosa de enunciados, código base y restricciones, así como una **supervisión docente** en aquellos casos en los que la corrección de las restricciones o la semejanza al código base resulten especialmente críticas.

6.4 Discusión y posibles mejoras

Los resultados de las pruebas permiten extraer algunas conclusiones sobre la **fiabilidad global** del sistema y sobre sus puntos débiles. En términos generales, la tubería técnica funciona de forma estable: *BIOTES* o el *fallback* producen evidencias de ejecución consistentes, la *nota_tests* refleja bien los casos superados o fallados y la retroalimentación generada por el modelo suele situar la *nota_ia* en un rango razonable respecto a la corrección manual. Los problemas detectados se concentran en aspectos más sutiles: la interpretación detallada de ciertas restricciones, la sensibilidad a cómo se formatea el contexto (enunciado, código base y restricciones) y algunos fallos esporádicos en la estructura de salida exigida.

Una primera línea de mejora pasa por cuidar la **redacción de los ejercicios** y de las restricciones que se envían al sistema. Las evidencias muestran que las restricciones largas, que agrupan varias ideas en una sola viñeta o que se formulan de manera ambigua, son más propensas a generar clasificaciones discutibles (*Cumple/No cumple/No verificable*). Resulta preferible descomponer las obligaciones en ítems más simples y específicos, evitar mezclar condiciones distintas en la misma frase y explicitar claramente qué parte es obligatoria y cuál es solo recomendación. De forma análoga, en el código base es útil incluir un comentario del tipo *# Continuar con la solución* tras el bloque proporcionado por el profesorado, de modo que el modelo distinga mejor qué se considera plantilla fija y qué corresponde al trabajo del alumnado.

En segundo lugar, las observaciones apuntan a posibles **ajustes en el prompt y en la lógica de la API**. El mensaje de sistema ya insiste en diferenciar obligaciones de recomendaciones y en marcar cada restricción como *Cumple*, *No cumple* o *No verificable*, pero se podría reforzar el requisito de citar fragmentos concretos de código como evidencia al justificar cada etiqueta, de modo que el juicio del modelo sea más trazable y verificable por el docente y el alumno.

En tercer lugar, los errores de **formato en la salida** sugieren que conviene seguir endureciendo las defensas en la API. Aunque la implementación actual ya limpia líneas no deseadas y busca de forma robusta la última línea que comienza por *NOTA_IA*;, seguiría siendo útil tolerar pequeñas variaciones del modelo (espacios extra, ligeros cambios de texto) e incluso contemplar estrategias de reintento automático cuando la nota no se puede extraer con fiabilidad. En cualquier caso, la experiencia del periodo de pruebas indica que estos fallos estructurales son poco frecuentes y que, cuando aparecen, reenviar la misma entrega suele producir una salida que sí respeta la plantilla establecida.

Por último, muchas de las limitaciones observadas están ligadas al **modelo de lenguaje concreto** utilizado (un modelo de la familia Mistral accedido a través de OpenRouter). La arquitectura de la API se ha diseñado para permitir cambiar de modelo con ajustes limitados en la configuración, lo que abre la puerta a evaluar en el futuro otras alternativas, incluidos modelos propietarios, que puedan ofrecer un comportamiento más robusto en la corrección de restricciones o en el mantenimiento del formato. Esta posible evolución se desarrolla con más detalle en la sección de *Conclusiones y líneas futuras*.

En conjunto, la discusión sugiere que el sistema es especialmente útil como herramienta de **retroalimentación inicial** y de apoyo en la evaluación, siempre que se diseñen enunciados, código base y restricciones con una estructura clara, y que se asuma cierta variabilidad en la interpretación detallada de las obligaciones más complejas. Sobre esta base, las mejoras

propuestas permiten aumentar la robustez del sistema sin perder de vista que la decisión definitiva sobre casos límite sigue correspondiendo al profesorado.

7

Conclusiones y líneas futuras

7.1 Conclusiones

Este Trabajo Fin de Grado nace de la necesidad de aliviar la **carga de corrección** en las asignaturas de programación y de ofrecer al alumnado una retroalimentación más rica que la mera información de “casos de prueba superados o fallados”. En muchos contextos docentes, **Virtual Programming Lab (VPL)** ya proporciona un entorno cómodo para entregar y ejecutar ejercicios, pero se centra en la evaluación automática por *tests* y deja en manos del profesorado la explicación detallada de errores y buenas prácticas. A partir de este problema se plantea integrar VPL con un **gran modelo de lenguaje (LLM)** ejecutado en una máquina local, de forma que VPL pueda comunicarse con dicho modelo, obtener una retroalimentación textual sobre la entrega del alumno y, a continuación, ceder el control a la corrección de casos de prueba propia de VPL, todo ello sin intervención adicional por parte del docente. El objetivo inicial era, por tanto, disponer de un sistema de corrección automatizada que respetara la privacidad de los datos y ofreciera comentarios útiles al estudiante directamente desde el entorno habitual de prácticas.

Durante el desarrollo del trabajo, esta idea inicial ha tenido que adaptarse a las **restricciones técnicas y de infraestructura** disponibles. Ejecutar un LLM completo en una máquina local personal no resultó viable por requisitos de hardware y despliegue. En su lugar, se ha optado por usar una **máquina de la UMA** que expone una API propia y se comunica con una pasarela de modelos en la nube (*OpenRouter*) para obtener la retroalimentación. Además, el primer *script* de evaluación bloqueaba a VPL e impedía que este ejecutara los casos de prueba con *BIOTES*; para resolverlo, se modificó el *script* de forma que fuera él mismo quien lance *BIOTES* y recoja sus resultados antes de enviarlos a la API.

Las pruebas de **integración** y la **evaluación experimental** con ejercicios reales muestran que, en la mayoría de los casos, el sistema se comporta de forma coherente: la *nota_tests* refleja adecuadamente los resultados de *BIOTES* o del *fallback*, la retroalimentación describe con detalle los aciertos y errores del código y la *nota_ia* suele situarse en un rango razonable

respecto a la corrección manual realizada por el autor del trabajo. Al mismo tiempo, el análisis de los errores observados revela fallos esporádicos en la estructura de la salida generada por el modelo, así como limitaciones claras a la hora de comprobar con detalle determinadas restricciones, derivadas de la sensibilidad del LLM al formato del contexto (enunciado, código base y restricciones).

En conjunto, las conclusiones apuntan a que el sistema es especialmente útil como herramienta de **retroalimentación inicial** y de apoyo a la corrección, pero no como **sustituto del docente** ni como mecanismo único de calificación. La presencia de alucinaciones puntuales en la evaluación del código proveniente del modelo probabilístico hacen desaconsejable adoptar directamente la nota devuelta por la API como calificación oficial del alumnado. En lugar de ello, la combinación de *nota_tests* y *nota_ia* debe entenderse como un indicador orientativo, que puede ayudar a priorizar qué entregas requieren una revisión manual más detallada y a proporcionar al estudiante **comentarios ricos** y estructurados sobre su código, manteniendo siempre la decisión final en manos del profesorado.

7.2 Líneas Futuras

El sistema desarrollado deja abierta una serie de líneas de trabajo que permitirían madurarlo y acercarlo más a un uso cotidiano en las asignaturas de programación. A partir de la experiencia de integración con VPL, de las pruebas realizadas y del análisis de errores, se pueden identificar varias direcciones razonables de evolución.

En el plano técnico, cabe seguir refinando la **representación del contexto** que se envía al modelo, en especial las **restricciones**. El trabajo actual ya muestra que restricciones largas o con varias ideas en la misma frase son más difíciles de verificar de forma fiable. Como línea futura, se podría explorar un formato más estructurado (por ejemplo, restricciones numeradas y breves, con campos separados para obligación y ejemplos) y estudiar empíricamente qué maneras de formularlas facilitan una evaluación más estable por parte del LLM.

Finalmente, el propio **componente LLM** admite varias mejoras futuras. El sistema actual se apoya en un modelo accesible a través de una pasarela de LLMs en la nube, seleccionado por su equilibrio entre coste y calidad. Como trabajo futuro, sería natural comparar de forma sistemática distintos **modelos** (abiertos y propietarios), evaluar su comportamiento en el contexto docente concreto y, si la infraestructura lo permite, explorar opciones de despliegue más privadas, como ejecutar un modelo en servidores de la UMA o utilizar pasarelas con garantías contractuales específicas sobre protección de datos. Este tipo de decisiones excede el alcance del presente TFG, pero constituyen un paso coherente para consolidar la solución en un entorno real de producción.

Apéndice A. Manual de Instalación

Este apéndice describe, paso a paso, el procedimiento necesario para desplegar y utilizar el sistema de retroalimentación automática basado en inteligencia artificial dentro de una actividad VPL en Moodle. Se parte completamente desde cero: sin API, sin configuración previa y sin ficheros añadidos a la actividad.

Los comandos utilizados en esta guía corresponden a una instalación realizada en una máquina Ubuntu, aunque el proceso es similar en cualquier sistema que permita ejecutar Python.

El proceso se divide en dos fases principales:

- Preparación de una máquina con **IP pública** para ejecutar la API.
- Configuración de una actividad VPL para enviar los datos a dicha API.

A.1 Requisitos previos

- Una máquina accesible por Internet con **IP pública**.
- Python 3.x instalado.
- Una API Key válida de OpenRouter (en este proyecto se utiliza siempre la misma).
- Un campus virtual creado con Moodle, con el plugin VPL instalado.
- Permisos de profesor o administrador para crear actividades VPL.

En este proyecto la API Key de OpenRouter es la misma para todas las instalaciones. Lo único que puede variar entre despliegues son la IP y el puerto desde los que se ejecuta la API.

A.2 Preparación de la máquina

A.2.1 Crear el directorio del proyecto

En la máquina donde se vaya a ejecutar la API se recomienda crear un directorio específico para el proyecto:

- `mkdir -p /tfg`
- `cd /tfg`

Dentro de dicho directorio se copia el archivo de la API, por ejemplo `ApiOpenRouter.py`.

A.2.2 Crear y activar el entorno virtual

Para aislar las dependencias del proyecto se utiliza un entorno virtual. Desde el mismo directorio anterior se crea y se activa el entorno con los siguientes comandos:

- `python -m venv .env`
(este paso solo es necesario la primera vez)
- `source .env/bin/activate`

Con el entorno virtual activado se instalan las dependencias necesarias, entre ellas:

- `pip install fastapi uvicorn requests`

A.2.3 Crear el archivo .env con la API Key

La API necesita conocer la clave `OPENROUTER_API_KEY`. Para ello se crea un archivo llamado `.env` en el directorio del proyecto.

Al escribir los siguientes comandos en la terminal, se creará el archivo y se permitirá introducir su contenido:

- Ejecutar: `cat > .env << 'EOF'`
(al copiar en la terminal, volver a escribir manualmente las comillas simples)
- En la línea siguiente escribir: `OPENROUTER_API_KEY=NUESTRA_API_KEY`
- Cerrar el bloque escribiendo: `EOF`
- Ajustar los permisos con: `chmod 600 .env`

Esto generará el archivo `.env` con la API Key necesaria para que la API funcione.

A.2.4 Ejecutar la API

Una vez activado el entorno virtual y creado el archivo `.env`, se puede ejecutar la API mediante el servidor `uvicorn`. Suponiendo que el archivo se llama `ApiOpenRouter.py` y que dentro de él se define la aplicación `app`, el comando de ejemplo es:

- `uvicorn --env-file .env ApiOpenRouter:app --host IP_PUBLICA --port 8000` (los guiones largos son dobles)

Debe sustituirse `IP_PUBLICA` por la IP del servidor. Tras ejecutarlo, la API quedará disponible en:

- `http://IP_PUBLICA:8000`

Mientras el proceso de `uvicorn` esté en marcha, la API estará lista para recibir peticiones desde VPL.

A.2.5 Comprobar el funcionamiento básico de la API

La API dispone de un punto de comprobación de estado. Para verificar que funciona correctamente, puede accederse a la siguiente dirección desde cualquier equipo con acceso a la red:

- `http://IP_PUBLICA:8000/health`

Si la API está activa, mostrará un mensaje de confirmación.

A.3 Configuración de la actividad VPL en Moodle

A.3.1 Creación de la actividad

1. Acceder al curso correspondiente en Moodle.
2. Seleccionar Añadir una actividad o contenido.
3. Elegir la actividad *Laboratorio Virtual de Programación (VPL)*.
4. Rellenar los campos básicos: nombre, descripción (enunciado), fechas, etc.
5. Guardar los cambios y acceder a los ajustes avanzados.

A.3.2 Añadir los ficheros necesarios

Dentro de los ajustes avanzados existe una sección llamada *Ficheros para la ejecución*. En ella deben añadirse los siguientes archivos:

- `vpL_evaluate.sh` (obligatorio, siempre el mismo).
- `vpL_evaluate.cpp` (obligatorio, siempre el mismo).
- `vpL_evaluate.cases` (obligatorio, contiene los casos del ejercicio).
- `enunciado.txt` (opcional).
- `codigo_base.txt` (opcional).
- `restricciones.txt` (opcional).

Podéis encontrar los archivos necesarios para una ejecución del sistema de forma completa, en [mi repositorio de GitHub](#). Hay que tener en cuenta de que los archivos: `vpL_evaluate.cases`, `enunciado.txt`, `restricciones.txt` y `codigo_base.txt` tienen un ejercicio básico dentro para probar el sistema, lo único que le faltaría es la solución.

A.3.3 Contenido del script `vpl_evaluate.sh`

El archivo `vpl_evaluate.sh` ejecuta los casos de prueba del alumnado, envía toda la información a la *API* y muestra la retroalimentación generada.

En la quinta línea del *script* aparece la dirección de la *API*, normalmente mediante la variable `API_URL`. Debe comprobarse que dicha dirección coincide con `http://IP_PUBLICA:8000` o con la que corresponda en el servidor.

A.4 Prueba de funcionamiento

Una vez configuradas la *API* y la actividad VPL, se recomienda realizar una prueba:

1. Acceder a la actividad VPL como estudiante.
2. Subir una solución sencilla.
3. Ejecutar la evaluación.
4. Confirmar que aparece un informe que combina:
 - los resultados de los casos de prueba (*BIOTES*).
 - la retroalimentación generada por la inteligencia artificial.

Si la retroalimentación no aparece:

- Verificar que la *API* está activa en `http://IP_PUBLICA:8000`.
- Revisar que la IP dentro de `vpl_evaluate.sh` es correcta.
- Comprobar que el archivo `.env` contiene la variable `OPENROUTER_API_KEY`.

Una vez superadas estas comprobaciones, el sistema queda listo para su uso, permitiendo que el alumnado reciba retroalimentación clara y personalizada sobre sus ejercicios.

Apéndice B. Manual de Usuario

B.1 Introducción

Este manual explica cómo utilizar el sistema de corrección automática integrado en **Moodle** mediante **VPL**, el *script* de evaluación del *Jail* y la **API** propia que se comunica con un modelo de lenguaje.

Se distinguen dos perfiles:

- **Profesor:** configura la actividad VPL, añade los ficheros del sistema y de cada ejercicio, y se asegura de que la API esté activa.
- **Estudiante:** edita y entrega su solución en VPL, ejecuta la evaluación y consulta la retroalimentación.

B.2 Perfil del profesor

B.2.1 Creación de una actividad VPL en Moodle

Para crear una nueva práctica de programación:

1. Acceder a Moodle con un usuario con rol de **profesor**.
2. Entrar en el curso correspondiente y activar la edición.
3. Seleccionar la opción de añadir actividad y elegir **Virtual Programming Lab (VPL)**.
4. Rellenar los campos básicos: nombre de la actividad, descripción y fechas de apertura/cierre (si se desean restricciones de entrega).
5. Guardar los cambios para crear la actividad.

A partir de aquí se deben añadir los ficheros que hacen funcionar el sistema.

B.2.2 Ficheros del sistema comunes

Todas las actividades que utilicen este sistema comparten dos ficheros principales:

- **vpl_evaluate.sh:** *script* de evaluación del *Jail*. Detecta el archivo de código del estudiante, compila y ejecuta el programa, intenta usar el *tester* oficial **BIOTES** si está disponible, ejecuta los casos de prueba y prepara los datos para la API.

- **vpl_evaluate.cpp**: programa auxiliar en C++ que se compila dentro del *Jail* como *tester* (**vpl_tester**) y se encarga de ejecutar y resumir los casos definidos en el fichero de casos.

Estos ficheros son comunes a todas las prácticas y normalmente no es necesario modificarlos ejercicio a ejercicio.

Para añadirlos a una actividad VPL:

1. Entrar en la actividad y acceder a los ajustes avanzados de la actividad.
2. Ir a la sección donde se gestionan los archivos de ejecución (ficheros para la ejecución).
3. Crear un archivo llamado **vpl_evaluate.sh** y pegar su contenido actualizado, o subirlo desde el equipo local. En la línea 5 de este archivo se debe cambiar la dirección IP por la dirección pública que se va a utilizar para la API.
4. Crear un archivo llamado **vpl_evaluate.cpp** y pegar su contenido, o subirlo desde el equipo local.
5. Guardar los cambios. El propio *script* se encarga de marcar el ejecutable correspondiente dentro del *Jail*.

El *script* **vpl_evaluate.sh** incluye, entre otros, estos comportamientos relevantes:

- Selecciona automáticamente el archivo de código del alumno buscando extensiones habituales (por ejemplo, .py, .java, .cpp, .c, .js, .ts).
- Si encuentra *vpl_evaluate.cases* y *vpl_evaluate.cpp*, compila un *tester* (**vpl_tester**) y ejecuta los casos al estilo de *BIOTES*.
- Si el *tester* oficial no está disponible, realiza una ejecución alternativa controlando la compilación y la ejecución de los casos.

B.2.3 Ficheros específicos de cada ejercicio

Además de los ficheros del sistema, cada ejercicio necesita sus propios ficheros de contexto didáctico:

- **enunciado.txt**: enunciado detallado del ejercicio.
- **codigo_base.txt**: código base o plantilla inicial que se ofrece al estudiante.
- **restricciones.txt**: lista de restricciones sobre la solución del ejercicio.
- **vpl_evaluate.cases**: definición de los casos de prueba del ejercicio (entradas, salidas esperadas y, si se desea, pequeña descripción).

Estos ficheros sí cambian de un ejercicio a otro. El flujo típico es:

1. En la misma sección de archivos de la actividad VPL, crear **enunciado.txt** y pegar el enunciado que se quiera utilizar tanto para el alumno como para la API.
2. Crear **codigo_base.txt** con la plantilla de código inicial (por ejemplo, funciones vacías o estructura mínima del programa).

3. Crear **restricciones.txt** y enumerar las restricciones pedagógicas que se quieren comprobar. Para el entendimiento del LLM las restricciones se deben estructurar en **Obligaciones** y **Recomendaciones**.

4. Crear **vpl_evaluate.cases** y definir los casos de prueba siguiendo el formato adoptado en el proyecto (una secuencia de bloques con identificador de caso, entrada y salida esperada).

En la práctica, una forma eficiente de crear nuevas prácticas es duplicar una actividad VPL ya configurada, mantener `vpl_evaluate.sh` y `vpl_evaluate.cpp`, y sustituir solo `enunciado.txt`, `codigo_base.txt`, `restricciones.txt` y `vpl_evaluate.cases`.

B.2.4 Puesta en marcha de la API con uvicorn

La API se implementa en Python, en el fichero **ApiOpenRouter.py**, y expone un servicio FastAPI que recibe los datos preparados por el *script* del *Jail* y devuelve la retroalimentación y la nota propuesta.

Antes de que los estudiantes utilicen el sistema, el profesor (o la persona responsable del servidor) debe:

1. Tener instalado Python y las dependencias necesarias de FastAPI, request y uvicorn en la máquina que va a estar ejecutando la API (normalmente mediante un archivo de requisitos del proyecto).
2. Crear un archivo **.env** en el mismo directorio que la **API**, con una línea que defina la clave de acceso al proveedor de modelos de lenguaje, por ejemplo:

```
OPENROUTER_API_KEY=sk-or-v1_TU_CLAVE_AQUI
```

3. Lanzar el servidor con un comando de la forma:

```
uvicorn --env-file .env ApiOpenRouter:app --host 0.0.0.0 --port 8000 --reload
```

El *script* **vpl_evaluate.sh** utiliza la variable **API_URL** para saber dónde enviar la petición; si no se define, asume ese valor por defecto.

Es recomendable revisar que:

- La API está escuchando en la dirección y puerto esperados.
- Desde el *Jail* de VPL se puede contactar con la API mediante la URL indicada en **API_URL**.

B.2.5 Comprobación rápida del sistema

Para verificar que todo funciona:

1. Crear o usar una actividad de prueba con todos los ficheros necesarios.
2. Entregar un programa sencillo que se sepa que compila y pasa algunos casos de prueba.
3. Ejecutar la evaluación en VPL y comprobar que:
 - Se ejecutan los casos de prueba y se muestra un resumen.
 - Aparece en la salida la retroalimentación generada por la API y la nota propuesta (nota total, nota de tests y nota de la IA).

Si algo falla, conviene revisar los logs del servidor uvicorn.

B.3 Perfil del estudiante

B.3.1 Acceso a la actividad y entrega

El flujo para el **estudiante** es el siguiente:

1. Iniciar sesión en Moodle y entrar en el curso correspondiente.
2. Abrir la actividad de tipo **VPL** asociada al ejercicio.
3. Dentro de la actividad, utilizar el editor integrado para escribir la solución o cargar el archivo de código desde el equipo.
4. Si el profesor ha definido un **código base**, este aparecerá ya precargado en el editor para ser completado.

Es importante respetar el lenguaje de programación indicado y seguir las instrucciones del enunciado.

B.3.2 Edición y reevaluación

El estudiante puede:

- Modificar su código tantas veces como sea necesario antes de la fecha límite.
- Guardar cambios intermedios en el editor de VPL.
- Volver a ejecutar la evaluación, si la actividad permite múltiples intentos.

La política de calificación (nota del último intento, mejor nota, etc.) depende de cómo haya configurado la actividad el profesor.

B.3.3 Evaluación y retroalimentación

Cuando el estudiante pulsa el botón de **Evaluar** en la interfaz de VPL:

1. El código se compila y se ejecuta en el *Jail*.
2. El sistema ejecuta los casos definidos en **vpl_evaluate.cases** y recoge las salidas.
3. El *script* prepara un resumen de resultados (incluyendo qué casos se han superado) junto con el enunciado, el código base y las restricciones.
4. Estos datos se envían a la API, que genera la retroalimentación y una nota propuesta.

Tras unos instantes, en la ventana de resultados de VPL se muestra:

- La salida estándar del programa y posibles mensajes de error.
- Un resumen de qué casos han pasado y cuáles han fallado.
- La **retroalimentación** generada por la IA, que explica los problemas detectados y sugiere mejoras.

- Una **nota propuesta**, junto con el desglose entre nota de tests y nota aportada por la IA.

Si la actividad permite varios intentos, el estudiante puede usar esta retroalimentación para corregir su código y volver a evaluar hasta mejorar su solución.

B.4 Problemas frecuentes y recomendaciones

B.4.1 Para el profesor

- Si no aparece retroalimentación de la API:
 - Comprobar que el servidor uvicorn sigue en ejecución.
 - Verificar que **API_URL** apunta a la dirección correcta.
- Si todas las entregas fallan en los mismos casos:
 - Revisar el contenido de **vpl_evaluate.cases** (entradas y salidas esperadas).
- Para crear nuevas prácticas de forma eficiente:
 - Duplicar una actividad ya configurada y cambiar solo enunciado.txt, codigo_base.txt, restricciones.txt y vpl_evaluate.cases.

B.4.2 Para el estudiante

- Si el programa no compila, revisar con calma los mensajes de error que muestra VPL.
- Si la evaluación tarda demasiado, comprobar que no haya bucles infinitos o esperas innecesarias en el código.
- Aprovechar siempre la **retroalimentación** para mejorar el diseño y la claridad del programa, no solo para alcanzar la nota mínima.

Bibliografía

- [1] B. Adhikari et al. "Comparing feedback from large language models and instructors". In: *Proceedings of the Eleventh ACM Conference on Learning @ Scale*. Association for Computing Machinery, 2024, pp. 335–339. DOI: 10.1145/3657604.3664660. URL: <https://doi.org/10.1145/3657604.3664660>.
- [2] I. Akhmetov et al. "Using GPT-4 to provide tiered, formative code feedback". In: *Proceedings of the 55th ACM Technical Symposium on Computer Science Education (SIGCSE 2024)*. Association for Computing Machinery, 2024, pp. 958–964. DOI: 10.1145/3626252.3630960. URL: <https://doi.org/10.1145/3626252.3630960>.
- [3] L. Arévalo et al. "Experiencia con una herramienta de pruebas de caja negra para el aprendizaje de asignaturas de programación en evaluación continua". In: *Actas de las XXIII JENUI*. 5–7 de julio de 2017. Cáceres, España, July 2017.
- [4] Cloudflare. ¿Qué es un LLM? <https://www.cloudflare.com/es-es/learning/ai/what-is-large-language-model/>. s. f.
- [5] DeepSeek. Sitio oficial y divulgación técnica. <https://www.xataka.com/basics/deepseek-que-como-funciona-que-opciones-tiene-esta-inteligencia-artificial>. 2025.
- [6] HostGator. Consulta la guía completa sobre el comando Bash. HostGator Blog. <https://www.hostgator.mx/blog/guia-sobre-comando-bash/>. 2025.
- [7] Instituto SISE. Características del lenguaje de programación Python. <https://www.sise.edu.pe/blog/caracteristicas-principales-python>. 2024.
- [8] Meta AI. Llama 3 / 3.1. <https://ai.meta.com/blog/meta-llama-3/>. 2024.
- [9] Microsoft. Evaluation metrics for LLM content. <https://learn.microsoft.com/en-us/ai/playbook/technology-guidance/generative-ai/working-with-llms/evaluation/list-of-eval-metrics>. 2024.
- [10] Microsoft. Visual Studio Code: Code editing. Redefined. <https://code.visualstudio.com/>. s. f.
- [11] Mistral AI. Documentación de Mistral. <https://docs.mistral.ai/>. s. f.
- [12] Mistral AI. Mistral AI. Documentation hub. <https://docs.mistral.ai/>. s. f.
- [13] Mistral AI. OpenRouter. mistralai/mistral-nemo – model card. <https://openrouter.ai/mistralai/mistral-nemo>. s. f.
- [14] OpenAI. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/>. 2025.
- [15] OpenRouter. OpenRouter. API Reference – Chat completion. <https://openrouter.ai/docs/api-reference/chat-completion>. s. f.

- [16] OpenRouter. *OpenRouter. Attribution headers*. <https://openrouter.ai/docs/app-attribution>. s. f.
- [17] Python Software Foundation. *El tutorial de Python*. URL: <https://docs.python.org/es/3.13/tutorial/>.
- [18] S. Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. s. f.
- [19] S. Tatham et al. *PuTTY Documentation Page*. <https://www.chiark.greenend.org.uk/~sgtatham/putty/docs.html>. 2025.
- [20] Uvicorn. *Uvicorn. Documentación oficial*. <https://www.uvicorn.org/>. s. f.
- [21] VPL. *VPL Jail System: introducción, instalación y configuración*. Documentación oficial de VPL. 2022–2025. Fuentes: <https://vpl.dis.ulpgc.es/documentation/vpl-jail-system-3.0.1/introduction.html>; <https://vpl.dis.ulpgc.es/documentation/vpl-jail-system-3.0.0/installation.html>; <https://vpl.dis.ulpgc.es/documentation/vpl-jail-system-2.7.0/configuration.html>. 2022.
- [22] VPL. *Manual oficial de VPL (documentación general y de casos BIOTES)*. <https://vpl.dis.ulpgc.es/documentation/vpl-3.4.3+/>. s. f.
- [23] Wikipedia. *FastAPI*. En *Wikipedia, la enciclopedia libre*. <https://es.wikipedia.org/wiki/FastAPI>. 2025.
- [24] Wikipedia. *Python*. En *Wikipedia, la enciclopedia libre*. <https://es.wikipedia.org/wiki/Python>. 2025.
- [25] Wikipedia. *Visual Studio Code*. En *Wikipedia, la enciclopedia libre*. https://es.wikipedia.org/wiki/Visual_Studio_Code. 2025.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga